Continual Learning from Engineering Principles

A (Rough) Theory of Resource-Bounded Incremental Induction

Daniel MacDonald*

February 17, 2026

*SingularityNET

Abstract

We argue that practical continual learning should be derived from explicit engineering desiderata rather than from mechanism-specific heuristics. Focusing on general online sequence prediction, we identify three interlocking requirements for high-utility continual learning systems: *robustness* to unsegmented and potentially pathological distribution shifts, *composability* with other clients with unknown data priors, and *resource efficiency* under fixed per-step compute and memory limits. Motivated by the incomputable ideal of Solomonoff induction and by incremental compression and program-synthesis lineages, we propose *Resource-Bounded Incremental Induction* (RBII): an online framework that maintains a bounded active pool of sandbox-valid predictor programs with query-limited access to an evolving state, updates their weights by Bayesian/exponential weighting, and discovers new predictors via a universal, resource-bounded search over *transformers* (program-to-program generators/edits). We introduce an operational, resource-aware conditional description length that quantifies how frozen code reduces the effective complexity of rediscovering predictors, yielding explicit multiplicative speedups in (re)acquisition time when regularities return. We then formalize a test-case notion of continual learning directly in terms of realized predictive performance, defining reacquisition delay and loss on recurring episodes, and show how RBII satisfies this criterion under mild assumptions. Studying a stricter variant without retrospective validation highlights an addressing bottleneck which must be overcome with additional heuristic mechanisms. RBII gives us an unbiased empirical perspective to discover and evaluate these mechanisms, qualifying it for the role of an engineering *framework*. Finally, we connect theory to practice by outlining an instantiation of RBII in a continual MNIST setting using a minimal DreamCoder-style DSL that integrates neural primitives as short accelerators while preserving a universal fallback.

Contents

1 Introduction **4**
1.1 Resource Bounded Incremental Induction 6
1.2 The rest of the paper 7

2 Bounded Conditional Induction with Reuse **7**
2.1 Hypotheses as conditional programs 8
2.2 Finite active pool and incremental Bayes/exp-weights 8
2.3 The discovery bottleneck and the reuse prior 8
2.4 Operational conditional complexity and telescoping gains 9
2.5 Universal transformer search and discovery time 9
2.6 Algorithm Sketch - Resource Bounded Incremental Induction 10

3 Performance on Continual Learning Test Cases **10**
3.1 A simple freezing policy (for analysis) 11
3.2 Regularities as resource-bounded randomness tests 11
3.3 What it means to have learned a regularity 12
3.4 When does a regularity "return"? Episodes and test cases 13
3.5 Relearning cost: delay and loss on a return episode 13

3.6 A basic mixture-dominance bound (post-entry) 14
3.7 Discovery, reuse, and expected reacquisition scaling 14
3.8 Obscurity and predictive value: when reuse should (and should not) help 15
3.9 Canonical continual-learning test cases and predicted behavior . . . 16
3.10 Summary: a test-case definition of continual learning 17

4 Dropping explicit validation: continual learning from state-only access 18
4.1 Why "freeze the best every step" is not sufficient by itself 18
4.2 State-conditioned operational description length 19
4.3 Breaking the addressing barrier 19
4.4 Restartable experts or a switching prior 20
4.5 Sufficient conditions for continual learning without validation 21
4.6 When continual learning is impossible under state-only access 22
4.7 Practical implications of the theory 22

5 Applications 23
5.1 Continual MNIST with RBII 24
5.1.1 A minimal DreamCoder-style DSL for predictors and transformers 24
5.1.2 State design: history + HNSW kNN + prototype→program recall 26
5.1.3 Illustrative predictor/transformer patterns 28
5.1.4 Walkthroughs: stable regimes, shifts, and returns 28
5.1.5 Limiting overhead in "good" regimes 30
5.1.6 Why heuristic retrieval does not become a single point of failure . 31
5.2 Lessons from Program Synthesis 32
5.2.1 Sophisticated Bootstrapping 32
5.2.2 Recursive Amortization 32

6 Related Work 34
6.1 Continual learning in modern machine learning 34
6.2 Universal induction, MDL, and resource-bounded schemes 34
6.3 More related work 37

7 Limitations 38

8 Future Work 39
8.1 Improved Formulation 39
8.2 Automated ML Engineering 39
8.3 Continual Predictive Coding in large scale systems 39
8.4 Generalized Amortization via Continual Predictive Caching 40
8.5 Cognitive Integration 40
8.6 Cognitive Orchestration 41

A Algorithm Details 43

1 Introduction

We argue for a principled approach to continual learning (CL) that derives from a set of specific engineering desiderata. We identify three interlocking properties that together give rise to practical and high-utility continual learning systems (CLS): *robustness*, *composability*, and *resource constraint*.

Our setting is general online sequence prediction, where we assume there is a sequence of observations and the system's job is to predict the next observation, observe it, and update some internal state in a way that improves subsequent predictions. A popular (though not fully general) way to characterize a system that "learns continually" is that the system learns from training data in a sequence of contexts or regimes and is *robust* to distribution shifts between regimes and is able to adapt its predictions to changing underlying characteristics of the sequence, but without forgetting its behavior in earlier regimes. We speculate that in one of the most fruitful application domains for CL systems, that of servicing cognitive processes with emergent behavior in AGI systems, the distribution shifts will become somewhat pathological from the point of view of any attempt to manually characterize them a priori. This leads us to view *composability*, or the ease with which we may connect a continual learning system with other systems, as a desirable property. Figures 1 and 2 depict the functional interface of a CLS and its interaction with a client.

Our third class of requirements concern resource efficiency, and it is the constraint that renders the first two interestingly non-trivial to achieve. Without resource constraints, robustness becomes trivially achievable: if we are using neural models, we could for example just retrain the whole model on an IID mixture of all the data seen so far. But such an approach is not feasible for real world applications such as incrementally growing a large language model on a situated stream of natural data or emergent cognitive objects. Moreover, it flies in the face of our lived exerience - humans somehow integrate information far more efficiently and effectively. Absent resource constraints, such a system is trivially composable, but with even modest global resource constraints, it cannot be leveraged (composed) effectively by any client. For example if we want to use an LLM to guide the expansion of productions in a reasoning system by detecting patterns in past successful derivations and narrow the search based on past experiences, a model that constantly retrains on all past data in order to maintain good prediction quality will quickly become too slow to be useful.

Continuing with the (simplistic) view of a learning system dealing robustly with distribution shifts, we observe optimal continual learning in this setting is subsumed by Solomonoff Induction: the best way, absent other information, to predict the next observation is to look for short programs that summarizes the whole sequence, including context shifts, and use their description lengths to weight their next output. From this view, an SI approach learns both the distributions and their shifts, but without needing to make any arbitrary distinction between them. In practice when engineering CL systems, many attempts have been made to use learning algorithms that fundamentally assume IID inputs and then duct-tape them together with other mechanisms to deal with distribution shifts. These heuristic attempts

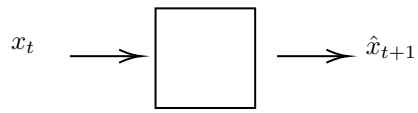

Figure 1: Functional interface of an online continual learning subsystem (CLS). A stream of individual observations x_t is provided and the system yields a prediction for the next observation x_{t+1}.

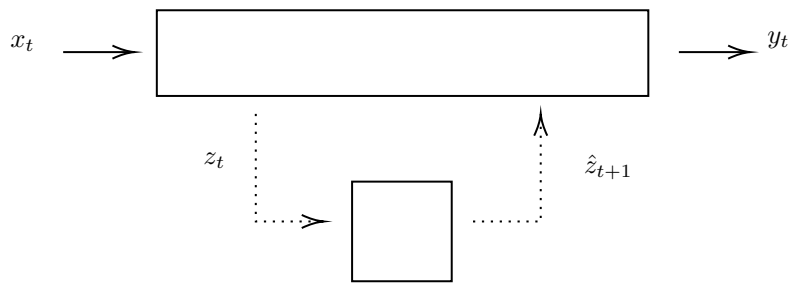

Figure 2: A CLS servicing a generic streaming client process. The client receives a stream of inputs x_t and provides a stream of outputs y_t. It derives some stream of finite representations z_t from x_t and (optionally) also some persistent client state. The CLS is provided z_t and yields its predictions $\hat{z}_{t+1}$ about subsequent representations. The arrangement will be fruitful if z_t exhibits learnable regularities, the client can leverage those predictions to improve its performance, and the utility of the predictions exceeds the cost of operating the CLS.

are often at odds with the actual character of real-world data we'd like to learn - distribution shifts of every conceivable type interleave constantly in e.g. real world audio and video streams from embodied agents, frustrating any fixed heuristics. The problem is no easier when dealing with internally generated dynamics of the cognitive processes of an intelligent agent, which makes it difficult or impossible to compose such a system with another system where the heuristic assumptions are not known or cannot be guaranteed to be met.

"But isn't all sequence prediction subsumed by Solomonoff Induction?" Of course, and this observation is usually unhelpful because although SI can be approximated, it's still extremely expensive to do so. Our contribution here is to suggest a principled path forward, drawing inspiration from historical and recent works on incremental induction ([Sch04]), incremental compression [Fra17], [FAS21], [FGL19]), and program synthesis ([EWN+20]). These works demonstrate promising theoretical and empirical results that suggest that rich, concise, and highly generalizable knowledge of the natural world is practically obtainable from iterating phases of program search with small inductive jump sizes and heavy reuse of previously learned primitives.

1.1 Resource Bounded Incremental Induction

We propose a framework for online continual sequence prediction which we dub Resource Bounded Incremental Induction (RBII). RBII maintains a small pool of online predictors (programs) which are allowed resource bounded query access to a (possibly growing) state which encodes the past. It reweights predictors based on their performance, keeping those that work and evicting those that fail in favor of new predictors. New predictors are constructed by resource-aware universal search - programs that construct new predictors are enumerated and evaluated, placed in the active pool if they seem promising. Predictors are general purpose programs in a generic DSL. Human prior knowledge is injected in the form of callable library code, specific state storage and query mechanism, or other resource-motivated constraints. Incremental learning hinges on the general character of incremental universal search – new programs can leverage old programs, and this allows the system to acquire rich conceptual knowledge in small, achievable steps. Continual learning behavior arises naturally from incremental induction – regularities that were well captured by old programs become cheaply available when they recur via program reuse.

RBII requires minimal assumptions about its input data stream, respects finite resource bounds for generating predictions and updating its state, and is robust to changes in the data stream's properties. Our framework makes a minimal number of high level heuristic choices and those choices are just those where the resource constraints require unavoidable assumptions about data stream properties. We distinguish our approach as resource-driven rather than mechanism-driven. The motivation for this is the lived experience of designing continual learning systems to scale: most proposed systems start from a set of heuristic mechanisms that seem neat and then the engineering effort is subsequently focused on compensating for their lack of scale under resource constraints or lack of robustness under changes in the underlying assumptions about the data. We hypothesize that starting from resource bounds and optimal universal learning mechanisms within those constraints will ultimately lead to more scalable systems and a far easier road for human engineers and their fledgling AGI companions to walk. To be clear, we do not claim to do away with all heuristics - indeed our analysis shows that we cannot and still expect meaningful performance. What we are proposing here is a different way of arriving at salient heuristics, one that is more aligned with our ideal engineering properties, allows for flexible tradeoffs between human and machine effort, and (we hypothesize) enables discovery of rich, emergent hybrid algorithms that would be cost prohibitive for either man or machine to derive alone.

Our system hinges on its universal properties but we do not expect it to "learn everything from scratch". A valuable aspect of our approach is that it allows us to liberally experiment with and introduce domain-specific biases (in the form of basic language primitives, callable high level heuristic processing modules, and opinionated state query mechanisms) without compromising the basic properties. This top-down approach contrasts with more traditional development methodologies which often couple specific heuristic assumptions to system architecture in a rigid and brittle fashion such that deviation from them is effectively unrecoverable

from within the system itself and difficult to remediate by the designer without major rewrites. If this statement seems overly strong, consider this: the assumption of IID training data is a heuristic one, directly coupled to the use of stochastic gradient methods for training neural networks. A satisfying design tweak that removes this assumption and still yields an efficient and effective gradient based training mechanism has been an explicit goal of AI researchers for decades and remains unsolved, despite its economic value being obvious to pretty much everyone (even Silicon Valley VCs) going back at least 10 years. In this sense, neural architectures are brittle and rigidly coupled - no low-entropy tweak has rescued them.

By maintaining universal flavor, our system avoids catastrophic failure in the face of the unexpected: it can always eventually predict any regularity that is capturable, modulo its resource constraints, and which pays for its detection cost in compression gain, suffering only temporarily the cost of over-weighting on the "wrong" initial biases. This property allows considerable design and experimentation flexibility and scalability, allowing human engineers to trade off compute for human effort as they see fit, including automating the experimental search for appropriate domain heuristics at multiple levels. The composability of systems following our framework opens the possibility of recursive self-improvement, which we explore only briefly here in section 5.2.2

1.2 The rest of the paper

Section 2 contains basic theoretical properties of interest and a high level algorithm sketch (section 2.6). Section 3 formalizes our test-case notion of continual learning and shows how RBII satisfies it. Section 4 analyzes a variant with even more severe constraints which reveals some interesting insight into the nature and role of memory systems in shaping learning under constraints. We show how to connect theory to practice in section 5.1 by showing a very crude but illustrative way of training an MNIST classifier in a continual manner. Related work is discussed in section 6. We wrap up with a summary of the limitations of this work and our scheme in section 7 and future work in section 8.

2 Bounded Conditional Induction with Reuse

We consider online prediction of an infinite binary stream $x_1, x_2, \dots \in \{0, 1\}$. The predictor maintains a state m_t updated incrementally by a fixed computable update rule

$$m_t = \mathrm{Upd}(m_{t-1}, x_t), \qquad m_0; \text{given}. \tag{1}$$

The state m_t may be unbounded in size, but predictive computation is bounded by requiring that any hypothesis/program interacts with m_t only through a structured query interface Q (e.g. random access, associative lookup, sketches), and each hypothesis evaluation is sandboxed by a fixed resource budget

$$\mathcal{R} = (q, T, \mathrm{mem}), \tag{2}$$

meaning at most q queries to Q, at most T compute steps, and bounded working memory. If a program exceeds $\mathcal{R}$ it is declared invalid. (Note that we allow m_t to

grow without bound because it simplifies the analysis and because our model is far less sensitive to storage costs than query costs.)

2.1 Hypotheses as conditional programs

A (sandbox-valid) predictor program p is a procedure that, when given oracle access to m through Q and constrained by $\mathcal{R}$, returns a probability $\pi_p(1 \mid m) \in [0,1]$ and $\pi_p(0 \mid m) = 1 - \pi_p(1 \mid m)$. At time t it induces a conditional distribution $\pi_p(\cdot \mid m_{t-1})$ for x_t.

The target of analysis is a fixed "true" conditional predictor $p^\star$ (or equivalently a true environment μ) that is realizable in the sandboxed hypothesis class:

$$\pi_{p^\star}(x_t \mid m_{t-1}) = \mu(x_t \mid x_{<t}) \quad \text{for all } t, \tag{3}$$

where m_{t-1} is the state produced by Upd on the realized past $x_{<t}$.

2.2 Finite active pool and incremental Bayes/exp-weights

At each time t the algorithm maintains a finite active set (pool) H_t of at most N sandbox-valid predictors. Each $p \in H_t$ has an unnormalized weight $W_{p,t} \geq 0$ updated by Bayes/exponential weights:

$$W_{p,t} = W_{p,t-1}\pi_p(x_t \mid m_{t-1}), \tag{4}$$

and the algorithm predicts using the normalized mixture

$$\hat{\pi}_t(1) = (1-\epsilon)\frac{\sum_{p\in H_t} W_{p,t-1}\pi_p(1 \mid m_{t-1})}{\sum_{p\in H_t} W_{p,t-1}} + \epsilon \cdot \tfrac{1}{2}, \tag{5}$$

with $\epsilon \in (0,1)$ providing a probability floor. The instantaneous log-loss (base 2) is

$$\ell_t = -\log_2 \hat{\pi}_t(x_t) \leq \log_2(2/\epsilon). \tag{6}$$

Once $p^\star$ enters H_t and is retained, standard mixture dominance yields bounded post-entry regret (formalized below).

2.3 The discovery bottleneck and the reuse prior

The central obstacle is that H_t is finite and $p^\star$ is not expected to be present initially. To ensure eventual discovery under a bounded per-step evaluation budget, we introduce a reuse mechanism inspired by "call/copy/edit" access to frozen code.

Let $\mathcal{S}_t = s_1, \ldots, s_{M_t}$ be a finite frozen store of programs discovered earlier and retained as read-only code. A new candidate is generated by a *transformer* program τ which is itself sandboxed by $\mathcal{R}$ and has read access to the codes of programs in $\mathcal{S}_t$ (and may query m_{t-1} through Q). The transformer outputs a predictor program p (or directly outputs predictions; we treat this as outputting a p).

We assign a prefix-code length prior to transformers:

$$w(\tau) \;\propto\; 2^{-|\tau|}. \tag{7}$$

The key point is that we impose no explicit compositional structure beyond universal program-to-program transformations: any computable manipulation of frozen code is permitted, but short transformers are exponentially preferred.

2.4 Operational conditional complexity and telescoping gains

Fix a reference interpreter $\mathsf{U}_{\mathcal{R}}$ enforcing the sandbox constraints and implementing the transformer interface. We measure description length in bits with respect to $\mathsf{U}_{\mathcal{R}}$.

Definition 1 (Resource-aware conditional description length)**.** *For a predictor program p and frozen store S, define*

$$K_{\mathcal{R}}(p \mid S) := \min_{\tau:\, \mathsf{U}_{\mathcal{R}}(\tau,S)=p} |\tau|. \tag{8}$$

Define $K_{\mathcal{R}}(p) := K_{\mathcal{R}}(p \mid \varnothing)$.

This quantity is operational: it measures the shortest sandbox-valid transformer needed to synthesize p given access to S.

Definition 2 (Marginal reuse gain and telescoping identity)**.** *Let $S_0 = \varnothing$ and $S_i = S_{i-1} \cup s_i$. Define the marginal gain for the target predictor $p^\star$ as*

$$\delta_i := K_{\mathcal{R}}(p^\star \mid S_{i-1}) - K_{\mathcal{R}}(p^\star \mid S_i) \geq 0. \tag{9}$$

Then the total gain after M freezes is the telescoping sum

$$K_{\mathcal{R}}(p^\star) - K_{\mathcal{R}}(p^\star \mid S_M) = \sum_{i=1}^{M} \delta_i. \tag{10}$$

This avoids any chain-rule assumptions about algorithmic mutual information: the identity holds by construction.

2.5 Universal transformer search and discovery time

We assume the algorithm devotes a fixed bounded "exploration" budget per time step to running transformers in a universal schedule (e.g. Levin-style dovetailing). Since each transformer evaluation is bounded by $\mathcal{R}$, each attempted transformer costs $O(1)$ wall-clock units. The schedule is chosen so that every transformer τ of length k is executed (at least once) within $O(2^k)$ exploration steps.

Assumption 1 (Universal transformer schedule)**.** *There exists a constant $c_{\mathrm{sch}} > 0$ such that for any transformer τ with $|\tau| = k$, the exploration process executes τ within at most $c_{\mathrm{sch}} 2^k$ attempted-transformer steps.*

Under this schedule, discovery time for $p^\star$ is controlled by $K_{\mathcal{R}}(p^\star \mid \mathcal{S}_t)$.

Proposition 1 (Discovery time scales exponentially in conditional description length)**.** *Fix a frozen store S. Let $\tau^\star$ be a shortest transformer achieving $\mathsf{U}_{\mathcal{R}}(\tau^\star, S) = p^\star$, so $|\tau^\star| = K_{\mathcal{R}}(p^\star \mid S)$. Under the universal transformer schedule, $p^\star$ is generated within at most*

$$c_{\text{sch}} 2^{K_{\mathcal{R}}(p^\star|S)} \tag{11}$$

attempted-transformer steps.

Consequently, any decrease of $K_{\mathcal{R}}(p^\star \mid S)$ by δ bits yields an immediate multiplicative speedup of at least 2^δ in the worst-case bound on additional discovery time.

2.6 Algorithm Sketch - Resource Bounded Incremental Induction

We present a high level sketch of our proposed Resource Bounded Incremental Induction algorithm here and give a more detailed version later in A.

Sketch: Resource Bounded Incremental Induction (RBII)

Initialize m_0, active pool H_0 (possibly empty), weights $W_{p,0}$, frozen store $S_0 = \varnothing$.
Fix a validation window of length w
for $t = 1, 2, \ldots$ **do**
  (Predict) Compute $\hat{\pi}_t(\cdot)$ from $(p, W_{p,t-1}) : p \in H_{t-1}$ with ϵ-smoothing; output $\hat{\pi}_t(1)$.
  Observe x_t and update state $m_t = \mathrm{Upd}(m_{t-1}, x_t)$.
  (Update weights) For each $p \in H_{t-1}$, set $W_{p,t} \leftarrow W_{p,t-1}, \pi_p(x_t \mid m_{t-1})$.
  (Explore) Execute a fixed bounded number of sandboxed transformers τ under the universal schedule; each transformer reads S_{t-1} and queries m_{t-1} via Q, outputs a candidate predictor p.
  (Select) If p is valid and scores well on a bounded validation window over recent values of x (using corresponding historical or recomputed m), insert p into the pool H_t (evicting lowest-weight elements to keep $|H_t| \leq N$ but never evicting the currently best-scoring predictor).
  (Freeze) If p passes a stricter "reusability" criterion (e.g. persistent advantage on validation), add p to the frozen store: $S_t \leftarrow S_{t-1} \cup p$.

3 Performance on Continual Learning Test Cases

What kind of data streams does our system learn on or fail to learn on? In what sense is that learning continual? In prior work on incremental compression [FAS21], it is observed that not all sequences are compressible incrementally and this motivated a subsequent effort to understand whether such a system would be useful in real-world scenarios. Franz argues that ubiquitous power-law correlations found in

nature are a consequence of branching compression hierarchies (BCH) which always admit short features and hence incremental compression [Fra17]. While a BCH-like prior is not a terrible fit for our setting, and we believe that any CLS should perform somewhat decently on BCH-originated streams, our resource-boundedness means that some long-range correlations latent in a BCH will not be extractable. At the same time, there are very non-BCH-like streams on which we expect excellent continual learning performance to both possible in principle and achieved by our scheme.

Functionally, we want a CLS to learn useful regularities and remember them when they recur. We'd prefer that it not learn noise, not forget what it learns when it learns new things, and be able to build upon what it learns in learning new things. All of these properties can be reduced to computational tests (and frequently are in the literature). Here we formalize these *test-case style* notions of continual learning directly in terms of predictive success on the realized stream. The guiding idea is the same one that underlies computable randomness tests: a sequence exhibits an "effective regularity" if some computable betting/compression strategy can systematically extract bits from it. Here, our "strategies" are precisely the sandbox-valid predictors, and our system's claim to continual learning is that (i) it can discover such regularities when they first occur, and (ii) it can *re-acquire* them with reduced delay and loss when they reappear, thanks to the frozen store and short transformers.

Throughout, we work with the same update rule $m_t = \mathrm{Upd}(m_{t-1}, x_t)$ and the same per-step resource constraint $\mathcal{R}$ as in Section 1.

3.1 A simple freezing policy (for analysis)

To isolate the effect of reuse, we analyze an intentionally simple freezing policy.

Assumption 2 (Always-freeze-the-incumbent policy)**.** *At each time t, after the (Select) step, the algorithm adds the current best-scoring predictor in the pool to the frozen store S_t (allowing duplicates, which do not affect conditional description lengths).*

This policy is not meant to be optimal as an implementation choice; it is a convenient upper envelope that guarantees that any predictor which becomes the incumbent at any time will be available for reuse later.

3.2 Regularities as resource-bounded randomness tests

Fix any sandbox-valid predictor program p. For an interval $I = [t_0 + 1 : t_1]$ (length $|I| := t_1 - t_0$), define the log-loss of p on the realized stream as

$$L_p(I) \;:=\; \sum_{t=t_0+1}^{t_1} -\log_2 \pi_p(x_t \mid m_{t-1}). \tag{12}$$

Let the algorithm's loss on I be

$$L_{\mathrm{alg}}(I) \;:=\; \sum_{t=t_0+1}^{t_1} \ell_t \;=\; \sum_{t=t_0+1}^{t_1} -\log_2 \hat{\pi}_t(x_t). \tag{13}$$

As a null baseline, the uniform predictor $\frac{1}{2}$ has loss $L_{1/2}(I) = |I|$.

Compression gain and "capital". Define the (realized) compression gain of p on I relative to the fair-coin baseline by

$$G_p(I) \;:=\; L_{1/2}(I) - L_p(I) \;=\; \sum_{t=t_0+1}^{t_1} \log_2\big(2\,\pi_p(x_t \mid m_{t-1})\big). \tag{14}$$

Equivalently, define a multiplicative "capital" process

$$\mathcal{M}_p(I) \;:=\; 2^{G_p(I)} \;=\; \prod_{t=t_0+1}^{t_1} 2\,\pi_p(x_t \mid m_{t-1}). \tag{15}$$

Intuitively, $G_p(I)$ counts how many bits p "saves" compared to guessing uniformly on that interval. Large positive $G_p(I)$ is the operational signal that a *p-regularity* is present on I.

Detectable regularities should pay for their description length. Since our discovery mechanism enumerates transformers with a prior $w(\tau) \propto 2^{-|\tau|}$, a natural test-case notion of "meaningful" regularity is that its compression gain exceeds its resource-aware description penalty.

Definition 3 (Detectable regularity relative to a store)**.** *Fix a frozen store S and a slack parameter $\beta \geq 0$. A predictor p is* (S, β)-detectable on I *if*

$$G_p(I) \;\geq\; K_{\mathcal{R}}(p \mid S) \;+\; \beta. \tag{16}$$

This mirrors the classical "randomness deficiency" viewpoint: a sequence segment I is non-random with respect to a class of tests if there exists a short test with large gain.

3.3 What it means to have learned a regularity

We want a notion of having learned a regularity that is (i) defined purely in terms of predictive performance on the realized stream, and (ii) compatible with finite-window validation and bounded computation.

Definition 4 (Excess loss and learning on an interval)**.** *For an interval $I = [t_0{+}1 : t_1]$ and a predictor p, define the algorithm's excess loss on I relative to p as*

$$\mathit{Reg}_p(I) \;:=\; L_{\mathrm{alg}}(I) - L_p(I). \tag{17}$$

Given a tolerance $\eta \geq 0$, we say the algorithm η-learns p on I *if*

$$\mathit{Reg}_p(I) \;\leq\; \eta\,|I|. \tag{18}$$

Thus, η-learning means that the algorithm's average log-loss on I is within η bits per symbol of the reference predictor p.

Learning time within an episode. When an episode begins at time t_0, define the (prefix) learning time

$$\tau_\eta(p;\, t_0) \;:=\; \min\Big\{n \in \mathbb{N} :\; \mathrm{Reg}_p([t_0+1 : t_0+n]) \le \eta n\Big\}, \tag{19}$$

with $\tau_\eta(p;\, t_0) = \infty$ if the set is empty. This captures the time it takes to begin matching p's performance at the level η.

3.4 When does a regularity "return"? Episodes and test cases

To talk about continual learning, we need a notion of the stream presenting the *same* regularity again later. We model this as recurring *episodes* on which a single sandbox-valid predictor is (approximately) the correct conditional model.

Definition 5 (Realizable episode and returns)**.** *An interval $I = [t_0+1 : t_1]$ is a* p-episode *if, for all $t \in I$, the data-generating conditional distribution satisfies*

$$\mu(x_t \mid x_{<t}) \;=\; \pi_p(x_t \mid m_{t-1}). \tag{20}$$

We say that p returns *if there exist infinitely many disjoint p-episodes in the stream.*

This is the simplest "task recurrence" model: tasks are predictors, and the world switches among them. More permissive notions (approximate realizability, drifting parameters) can be treated by allowing $\mathrm{KL}(\mu(\cdot \mid x_{<t}) \| \pi_p(\cdot \mid m_{t-1}))$ to be small, but we keep the exact case for clarity.

3.5 Relearning cost: delay and loss on a return episode

Fix a returning predictor p and consider its r-th return episode $I^{(r)} = [t_r+1 : t_r+L_r]$. The continual-learning desideratum is that after the first time p is learned, subsequent returns incur smaller *delay* and smaller *excess loss*.

Definition 6 (Reacquisition delay and reacquisition loss)**.** *Fix $\eta \ge 0$. On the return episode $I^{(r)} = [t_r+1 : t_r+L_r]$, define the reacquisition delay*

$$D_\eta^{(r)}(p) \;:=\; \min\{\tau_\eta(p;\, t_r),\; L_r\}, \tag{21}$$

and the reacquisition loss (excess loss paid before reacquisition) by

$$C_\eta^{(r)}(p) \;:=\; \mathit{Reg}_p([t_r+1 : t_r+D_\eta^{(r)}(p)]). \tag{22}$$

If $D_\eta^{(r)}(p) \ll L_r$ and $C_\eta^{(r)}(p)$ is small for large r, we interpret this as the system having retained and reused the regularity associated with p.

3.6 A basic mixture-dominance bound (post-entry)

The next lemma isolates the "easy" part of continual learning: once a good predictor is in the pool with nontrivial weight and is retained, the mixture incurs only a bounded overhead relative to it.

Let $w_{p,t} := \frac{W_{p,t}}{\sum_{q \in H_t} W_{q,t}}$ denote p's normalized weight when $p \in H_t$.

Lemma 1 (Mixture dominance with ϵ-smoothing)**.** *Fix an interval $I = [t_0+1 : t_1]$. Suppose that $p \in H_{t_0}$ and that p is never evicted during $t = t_0, \ldots, t_1 - 1$. Then the algorithm's cumulative loss on I satisfies*

$$L_{\text{alg}}(I) \;\le\; L_p(I) \;+\; \log_2\Big(\frac{1}{w_{p,t_0}}\Big) \;+\; |I| \log_2\Big(\frac{1}{1-\epsilon}\Big). \tag{23}$$

Equivalently,

$$Reg_p(I) \;\le\; \log_2\Big(\frac{1}{w_{p,t_0}}\Big) \;+\; |I| \log_2\Big(\frac{1}{1-\epsilon}\Big). \tag{24}$$

Proof. Let $\tilde{\pi}_t$ denote the normalized mixture over H_{t_0} without smoothing. Then $\hat{\pi}_t(x_t) \ge (1-\epsilon)\tilde{\pi}_t(x_t)$, so $-\log_2 \hat{\pi}_t(x_t) \le -\log_2 \tilde{\pi}_t(x_t) + \log_2 \frac{1}{1-\epsilon}$. Summing over $t \in I$ and applying standard mixture dominance $-\sum_{t \in I} \log_2 \tilde{\pi}_t(x_t) \le L_p(I) + \log_2 \frac{1}{w_{p,t_0}}$ yields the claim. □

Lemma 1 shows that if a returning task's predictor can be kept in the pool, the remaining question is how quickly it (or an equivalent predictor) is *rediscovered* when it is not already active.

3.7 Discovery, reuse, and expected reacquisition scaling

We now connect the operational continual-learning metrics above to the reuse mechanism through $K_{\mathcal{R}}(\cdot \mid S)$.

Let $E_{\text{exp}} \in \mathbb{N}$ be the (constant) number of attempted-transformer executions performed per time step in the (Explore) phase. Under Assumption 2 the store S_t grows monotonically.

Weight injection for newly generated programs. To make the post-entry bound depend cleanly on transformer length, we assume a simple "prior-consistent" initialization rule when a transformer τ generates a candidate p.

Assumption 3 (Prior-consistent weight injection)**.** *When a transformer τ of length $|\tau|$ generates a valid predictor p, the algorithm initializes its weight at insertion time t as*

$$W_{p,t} \;\leftarrow\; 2^{-|\tau|} \sum_{q \in H_t} W_{q,t}, \tag{25}$$

before any subsequent normalization. (If p is already in the pool, the rule is skipped.)

Under this rule, p enters with normalized weight $w_{p,t} \ge \frac{2^{-|\tau|}}{1+2^{-|\tau|}}$, hence $\log_2(1/w_{p,t}) \le |\tau| + 1$.

Proposition 2 (Reacquisition delay scales exponentially in conditional description length)**.** *Fix a frozen store S and a returning task predictor p. Let $k := K_{\mathcal{R}}(p \mid S)$ and let $\tau^\star$ be a shortest transformer achieving $\mathsf{U}_{\mathcal{R}}(\tau^\star, S) = p$ (so $|\tau^\star| = k$). Under the universal transformer schedule (Assumption 1) and E_{exp} transformer executions per time step, the time (in environment steps) until p is generated is at most*

$$D_{\text{gen}}(p \mid S) \;\le\; \left\lceil \frac{c_{\text{sch}}}{E_{\text{exp}}}\, 2^k \right\rceil. \tag{26}$$

Theorem 1 (Episode regret decomposes into a pre-discovery and post-entry term)**.** *Consider a p-episode $I = [t_0 + 1 : t_0 + L]$ with frozen store S_{t_0} at its start and let $k := K_{\mathcal{R}}(p \mid S_{t_0})$. Assume: (i) the exploration/selection mechanism inserts p into the pool immediately upon generation and does not evict it during the remainder of the episode; and (ii) weights are injected as in Assumption 3. Then the algorithm's excess loss on I satisfies*

$$Reg_p(I) \;\le\; D_{\text{gen}}(p \mid S_{t_0}) \cdot \log_2(2/\epsilon) \;+\; (k+1) \;+\; L \log_2\!\left(\frac{1}{1-\epsilon}\right), \tag{27}$$

where $D_{\text{gen}}(p \mid S_{t_0})$ is as in Proposition 2.

Proof. Split I into the prefix before p is generated and the suffix after p enters. On each pre-generation step, $\ell_t \le \log_2(2/\epsilon)$, so the prefix contributes at most $D_{\text{gen}}(p \mid S_{t_0}) \log_2(2/\epsilon)$. At insertion time, Assumption 3 gives $\log_2(1/w_p) \le k + 1$. Apply Lemma 1 on the suffix to obtain the post-entry term. □

Immediate speedups from reuse. Theorem 1 makes the "continual" aspect explicit: whenever the frozen store reduces the conditional description length of a returning predictor by δ bits, the worst-case additional discovery-time bound improves by a factor 2^δ.

Corollary 1 (Reuse gains translate into multiplicative reacquisition speedups)**.** *If, at some time t_0, the store changes from S to S' and $K_{\mathcal{R}}(p \mid S) - K_{\mathcal{R}}(p \mid S') \ge \delta$, then*

$$D_{\text{gen}}(p \mid S') \;\le\; 2^{-\delta}\, D_{\text{gen}}(p \mid S) \quad \textit{(up to the same constants).} \tag{28}$$

3.8 Obscurity and predictive value: when reuse should (and should not) help

The bounds above isolate two distinct obstacles:

1. **Search/description obstacle:** generating a suitable predictor scales as $2^{K_{\mathcal{R}}(p|S)}$ (Proposition 2).
2. **Statistical/utility obstacle:** even if a candidate predictor is generated, a finite validation window may not reliably distinguish it from incumbents unless it provides enough gain.

The second point is what makes "rare and weak" regularities hard to learn (and arguably not worth freezing).

To capture predictive value, define the expected per-symbol gain of p on a p-episode:

$$\gamma(p) \;:=\; \frac{1}{L}\,\mathbb{E}\big[G_p(I)\big] \;=\; 1 - \frac{1}{L}\mathbb{E}\big[L_p(I)\big] \quad \in [0,1], \tag{29}$$

where the expectation is over the episode randomness under $\mu = \pi_p$.

Proposition 3 (Detectability requires episode length to pay the description penalty)**.** *Fix a store S and a predictor p with $k = K_{\mathcal{R}}(p \mid S)$. If a p-episode has length L and expected gain rate $\gamma(p)$, then*

$$\mathbb{E}\big[G_p(I)\big] \;=\; \gamma(p)\,L. \tag{30}$$

In particular, if $\gamma(p)\,L < k$, then p is not *$(S,0)$-detectable on the episode in expectation (Definition 3), so any MDL/prior-penalized validation procedure will have insufficient evidence to justify selecting (and hence freezing) p.*

Proposition 3 formalizes the intuition in the sketch: an "obscure" regularity can mean either large conditional description length k (hard to generate) or small gain rate $\gamma(p)$ and/or short episode length L (hard to validate). Only when the stream presents enough *bits of evidence* ($\gamma(p)L$) can a short transformer-enabled predictor be selected and then reused.

3.9 Canonical continual-learning test cases and predicted behavior

We now describe several stylized test cases, phrased entirely in terms of returning predictors and episode structure, together with the behavior predicted by the bounds above.

Test case A: recurring finite task family ("context switching"). Let $\{p^{(1)}, \ldots, p^{(K)}\}$ be a finite set of sandbox-valid predictors. The stream consists of episodes, each generated by exactly one $p^{(i)}$, and each $p^{(i)}$ returns infinitely often (Definition 5). Assume each episode length L is large enough that the episode is (S,β)-detectable for some modest β once the appropriate $p^{(i)}$ can be generated.

Prediction. The first time each $p^{(i)}$ appears, acquisition is bottlenecked by $2^{K_{\mathcal{R}}(p^{(i)})}$. Under Assumption 2, once $p^{(i)}$ becomes incumbent it is added to the frozen store; thereafter, when $p^{(i)}$ returns, its conditional description length drops substantially (often to $O(1)$ up to addressing overhead), and Theorem 1 predicts near-immediate reacquisition and $O(1)$ per-episode excess loss.

Test case B: compositional curriculum ("edit distance between tasks"). Let $p^{(1)}, p^{(2)}, \ldots$ be a sequence of task predictors appearing in order. Suppose that each new task is an "edit" of past code, in the operational sense that

$$K_{\mathcal{R}}\big(p^{(j)} \mid S_{t_{j-1}}\big) \;\ll\; K_{\mathcal{R}}\big(p^{(j)}\big), \tag{31}$$

where t_{j-1} is a time after which earlier tasks have been frozen. (E.g., a short transformer loads a previous predictor and patches a constant, swaps a subroutine, or adds a small gating rule.)

Prediction. Discovery of later tasks scales as $2^{K_{\mathcal{R}}(p^{(j)}|S)}$ rather than $2^{K_{\mathcal{R}}(p^{(j)})}$, giving large speedups whenever reuse gains are positive. This is precisely the setting where the telescoping identity for marginal reuse gains (Definition 2) becomes operational: each frozen program can reduce future conditional description lengths, and the resulting speedups multiply.

Test case C: rare, low-value regularities ("glimpses"). A predictor p appears only in short episodes (small L) and/or has small gain rate $\gamma(p)$. Even if p is simple, Proposition 3 says these episodes may fail to pay $K_{\mathcal{R}}(p \mid S)$.

Prediction. The algorithm will not reliably select or freeze p (and should not, if the goal is to prioritize reusable structure). When p later reappears, reacquisition may again be slow. This is a failure mode only if one expects "one-shot" memorization of weak patterns; it is also a desirable robustness property if such patterns are noise.

Test case D: non-recurrent drift ("no tasks return"). If the stream never presents the same predictive regularity twice (in the sense of Definition 5), then the frozen store cannot provide conditional complexity reductions for future tasks.

Prediction. RBII reduces to bounded-compute universal search plus exp-weights within the active pool; performance is governed by first-discovery costs and finite-pool tracking, with no systematic reduction in reacquisition delays (because there are no reacquisitions).

3.10 Summary: a test-case definition of continual learning

The above formalization suggests an operational criterion for continual learning under bounded compute:

> A system exhibits continual learning on a stream if, for a broad class of returning predictors p, the reacquisition delay $D_{\eta}^{(r)}(p)$ and reacquisition loss $C_{\eta}^{(r)}(p)$ (Definition 6) are substantially smaller on later returns than on first presentation, and this improvement can be explained by decreases in $K_{\mathcal{R}}(p \mid S_{t_r})$ induced by the frozen store.

This criterion is independent of any particular generative prior: it only asks whether the system can cache *useful predictive code* in a form that is accessible to short, sandbox-valid transformers, and whether that cached code yields measurable, repeatable reductions in discovery time and log-loss when the same regularities recur.

Note that the while very simple model of returning episodes is not fully general, we still get quite a bit of leverage out of it and the operational criteria above already captures and generalizes popular metrics of continual learning performance used

in the literature. We leave the development of a fully general theory of test-case continual learning for future work.

4 Dropping explicit validation: continual learning from state-only access

In this section we explore a variant of RBII that we find instructive to study and also tackle the subtle but important issue of addressability of past programs. We start by *removing* any fixed-window (or any other) retrospective validation. New candidates are never re-checked on past data; their only evaluation signal is their online log-loss going forward. Moreover, the only access to the past available to *predictor programs* is via bounded queries to the evolving state m_{t-1} through Q. We keep the aggressive policy of freezing the incumbent predictor at every step (which is probably not optimal, but pressure-tests the arguments).

The question is: does this still yield continual learning in the test-driven sense of Section 3 (small reacquisition delay/loss on returns), and if not, what additional assumptions or mechanisms are required?

4.1 Why "freeze the best every step" is not sufficient by itself

Freezing produces a large read-only store $S_t = \{s_1, \ldots, s_{M_t}\}$, with $M_t \approx t$ under "freeze every step". This guarantees that any predictor that ever becomes best is eventually present as code in S_t. However, *presence in the store is not the same as accessibility under bounded compute.*

The basic obstruction is an *addressing bottleneck*: if the only way to reuse a stored program is to specify its identity by an explicit index or name inside the transformer code, then the shortest transformer that retrieves a particular old program typically has to encode $\Omega(\log M_t)$ bits just to pick it out among M_t possibilities. Under a universal schedule, this yields rediscovery times that grow at least linearly with M_t.

Proposition 4 (Addressing barrier without state-indexed recall)**.** *Assume that the only way a transformer can select an element of S_t is by hard-coding an identifier that uniquely specifies it among M_t stored programs, and that this identifier is not obtainable from the current state m_{t-1} via Q (i.e., it must be written into the code of the transformer). Then for a typical stored program $p \in S_t$,*

$$K_{\mathcal{R}}(p \mid S_t) \;\gtrsim\; \log_2 M_t, \tag{32}$$

so under the universal transformer schedule the worst-case additional rediscovery time satisfies

$$D_{\mathrm{gen}}(p \mid S_t) \;\gtrsim\; 2^{K_{\mathcal{R}}(p|S_t)} \;\gtrsim\; M_t \;\approx\; t. \tag{33}$$

In particular, reacquisition delay on returns can grow without bound with absolute time t, which violates the intended "continual" behavior for long-running streams.

Interpretation. With no retrospective validation, the store can only help if the system can *retrieve* the right old code quickly from the *current* state. Otherwise, freezing every step turns into a needle-in-a-haystack problem.

4.2 State-conditioned operational description length

The scenario in this section forces a distinction that was implicit earlier: transformers run at time t have oracle access not only to the store S_{t-1} but also to the *current* state m_{t-1} via Q. This motivates a state-conditioned operational description length.

Definition 7 (State-conditioned resource-aware description length)**.** *Fix time t and write $m := m_{t-1}$. Define*

$$K_{\mathcal{R}}(p \mid S; m) \;:=\; \min_{\tau:\ \mathsf{U}_{\mathcal{R}}(\tau, S; \mathsf{Q}_m) = p} |\tau|, \tag{34}$$

where $\mathsf{U}_{\mathcal{R}}$ executes τ with read access to S and oracle access to m through Q (under the sandbox budget $\mathcal{R}$), and outputs a predictor program p.

This quantity captures what actually matters when validation is disallowed: the speed of *contextual recall* from the current state.

4.3 Breaking the addressing barrier

To beat Proposition 4, the system must avoid encoding a large store index in the transformer code. In other words, the system must have a way to construct relevant retrieval indices more cheaply. Certain indices will naturally be short (because some addresses have lower algorithmic complexity than others), but these naturally likely indices do not necessarily correspond to useful programs.

In this section we will engage a particular heuristic bias and see how it impacts our test-case formulation. The point of this section is to illustrate an example from a broad class of possible mechanisms rather than outline a general theory of them just yet. In the subsequent discussion in section 4.7, we'll connect this exercise with a broader characterization of RBII as an engineering framework.

The workaround we'll try is to have the state m_t itself maintain *a retrievable index* into the frozen store. Concretely, we assume m_t contains an associative structure that maps bounded-compute "keys" of the current situation to pointers into S_t. We begin by characterizing a situational recall mechanism:

Assumption 4 (Self-indexing recall structure inside m)**.** *There exist sandbox-valid query procedures*

$$\mathsf{Key}(\cdot),\ \mathsf{Recall}(\cdot)$$

implementable using $O(1)$ queries to Q and $O(1)$ compute under $\mathcal{R}$, such that:

1. ***(Key)*** $\mathsf{Key}(m)$ *outputs a bounded-length key $c \in \{0,1\}^{\kappa}$ for some fixed κ (or more generally a key in some finite domain).*

2. *(**Recall**)* $\mathsf{Recall}(m)$ *outputs a list of at most K pointers to programs in the frozen store S (e.g. $K = O(1)$), obtained by associative lookup using $c = \mathsf{Key}(m)$.*

3. *(**Write-on-freeze**) Whenever a program p is frozen at time t, the update mechanism writes p into the recall structure under the current key $c_t = \mathsf{Key}(m_{t-1})$, so that future calls to* Recall *on similar states can return p.*

Remark. Assumption 4 does *not* require any replay of past bits. It only requires that the update rule Upd maintain a content-addressable index (potentially unbounded in size) that is accessible through the bounded query interface Q.

Note that practical indexing mechanisms tend to exhibit slow $\Theta(\log t)$ growth in compute cost with number of environment steps. This factor can be absorbed into a time varying sandbox resource bound $\mathcal{R}_t$ if desired without changing the overall structure or implications of what follows, but to keep things clean we stick with the $O(1)$ assumption.

Proposition 5 (Constant-length recall transformer)**.** *Under Assumption 4, there exists a constant-length transformer τ_{rec} (independent of t and $|S_t|$) that: (i) queries m to obtain pointers* $\mathsf{Recall}(m)$*, and (ii) outputs the referenced program(s) from S (or a wrapper that calls them). Consequently, for any p that is returned by* $\mathsf{Recall}(m)$*,*

$$K_{\mathcal{R}}(p \mid S; m) \;\le\; |\tau_{\mathrm{rec}}| \;=\; O(1), \tag{35}$$

and under the universal transformer schedule p can be regenerated within $O(1)$ exploration attempts (up to the fixed scheduling constant).

Interpretation. Once the state can recall relevant code, the "reuse prior" becomes meaningful again: the description length controlling discovery is no longer $\log |S_t|$ (which grows), but the constant length of a generic "recall-and-copy" transformer.

4.4 Restartable experts or a switching prior

Even with perfect recall, a second obstruction remains: in a nonstationary stream with recurring tasks, plain Bayes/exp-weights over experts can suffer from *posterior inertia*. An expert that was good long ago but performs poorly in between can have its weight driven exponentially small, and when the task returns it may take many steps of advantage to recover.

Since we have removed validation, the cleanest solution is to allow *restarts*: when recall suggests a previously good predictor, we insert a *fresh instance* with a fresh prior weight, rather than inheriting the old (potentially tiny) weight.

Assumption 5 (Restart-on-recall weight injection)**.** *Whenever the exploration phase produces a predictor p (in particular via τ_{rec}), the algorithm inserts a new instance of p into the pool with initial normalized weight at least $w_{\min} > 0$ (or, equivalently, with unnormalized weight at least a fixed fraction of total mass). This injection is allowed even if an older instance of p exists in the pool.*

Assumption 5 is operationally equivalent to imposing a *switching prior* (or "fixed share") at the meta-level: some fixed fraction of mass is continuously allocated to freshly (re)instantiated hypotheses.

4.5 Sufficient conditions for continual learning without validation

We can now state a clean test-driven condition under which the system still learns continually.

Consider a returning predictor p and a return episode $I^{(r)} = [t_r + 1 : t_r + L_r]$ that is realizable by p in the sense of Definition 5. We need two properties: (i) *recognizability from state* (recall actually returns p or an equivalent predictor); and (ii) *separation* (once present with nontrivial weight, p dominates quickly on its episode).

Assumption 6 (State-recognizable returns)**.** *There exists a burn-in $B_0 = O(1)$ such that for every return episode $I^{(r)}$ of p and every $t \in [t_r + B_0 : t_r + L_r]$,*

$$p \in \mathsf{Recall}(m_{t-1}), \tag{36}$$

i.e. the recall mechanism returns p (or a predictor with identical conditionals to p on the episode) from the current state.

Assumption 7 (Per-step separation on the return episode)**.** *There exists $\Delta > 0$ such that on any p-episode and for any competing predictor q in the active pool,*

$$\mathbb{E}\Big[-\log_2 \pi_q(x_t \mid m_{t-1}) + \log_2 \pi_p(x_t \mid m_{t-1}) \;\Big|\; x_{<t} \Big] \;\geq\; \Delta \qquad \text{for all } t \in I^{(r)}. \tag{37}$$

Equivalently, the conditional KL gap satisfies $\mathrm{KL}(\pi_p(\cdot \mid m_{t-1}) \| \pi_q(\cdot \mid m_{t-1})) \geq \Delta$ *on the episode.*

Theorem 2 (Continual learning without validation via recall + restarts)**.** *Assume: (i) a self-indexing recall structure (Assumption 4), (ii) restart-on-recall weight injection (Assumption 5), (iii) state-recognizable returns (Assumption 6), and (iv) per-step separation on each return episode (Assumption 7). Then for every return episode $I^{(r)} = [t_r + 1 : t_r + L_r]$ with L_r sufficiently large, the reacquisition delay and reacquisition loss (Definition 6) satisfy, with high probability,*

$$D_\eta^{(r)}(p) \;\lesssim\; B_0 \,+\, O\Big(\frac{1}{\Delta} \log_2 \frac{1}{w_{\min}}\Big), \qquad C_\eta^{(r)}(p) \;\lesssim\; O\Big(\log_2 \frac{1}{w_{\min}}\Big) \,+\, O(1), \tag{38}$$

up to the same additive ϵ-smoothing overhead terms as in Lemma 1. In particular, these bounds do not grow with the step time t_r or with the length of the gap between returns.

Interpretation. Theorem 2 recovers the intended continual-learning behavior *without any replay/validation*, but only after adding two algorithmic ingredients:

1. **State-indexed recall:** m must cache pointers from current contexts to relevant frozen code, so that reuse is accessible with constant-length transformers (Proposition 5).

2. **Restart/switching mechanism:** the online weighting must allow previously good code to re-enter with nontrivial mass when recalled, avoiding posterior inertia from long intervening periods (Assumption 5).

4.6 When continual learning is impossible under state-only access

Finally, it is worth making explicit a fundamental limitation in this regime. If two distinct "tasks" are not distinguishable from the current state m under bounded queries, then no system that only consults m (and does not replay the past) can reliably decide which stored predictor to reuse.

Concretely, if there exist two predictors $p \neq q$ and two long intervals on which the joint process (m_{t-1}, x_t) induced by p is (approximately) indistinguishable from that induced by q under any $\mathcal{R}$-bounded test, then *no* recall mechanism $\mathsf{Recall}(m)$ can consistently return the right one, and continual learning (in the sense of uniformly small reacquisition delay on returns) is unachievable without enriching the state m or relaxing the access constraints.

4.7 Practical implications of the theory

In the foregoing setup, looking harder at addressing costs and removing fixed validation windows lead us to try a heuristic that explicitly pushed long-term learning into the design of the state m and its query interface Q. In our initial presentation (section 1.1), a fixed validation window ensured that candidate weights were well-behaved, but did not deal with addressing costs directly. We informally observe that to support test-driven continual learning, the detailed instantiation of RBII must (at minimum) enable: (i) returning predictors p to be addressed with fewer than $\log t$ bits via transformer search (ii) a restart/switching prior so recalled code gets fresh weight, and (iii) eviction/protection rules that ensure recalled candidates are actually evaluated online long enough to accrue evidence (e.g. a small "protected" set of recently recalled experts). Points (ii) and (iii) are relatively mild conditions which both variants deal with in different ways without much trouble, so we don't anticipate the fuzziness in the theory here to be a serious obstacle in practice.

Point (i) is non-trivial and cuts to the core of what RBII is and how it works: we need to overcome the addressing barrier in order to expect meaningful continual learning to happen. How do we do this? Well, we might just try letting the system run for a long time or with a lot of resources and see what happens. This isn't a terrible idea - because our candidate programs are universal, they can in principle discover and reuse any heuristic that could be implemented in the DSL, if it produced enough predictive gain. However it's not guaranteed that this will happen under tight resource constraints, and in practice likely will need to leverage some augments we aren't trying to formally integrate yet (e.g. library compression techniques, section 5.2).

Another idea is to just try putting in a special memory mechanism and the DSL calls to support it, and see what happens. This isn't a terrible idea either - because

we aren't *forcing* the system to use the primitive, the cost of using it is balanced fairly against the cost of other mechanisms available through short transformer search.

Of course we can combine both approaches - we can equip the system with lots of different possibly useful primitives and retrieval indexing mechanisms, and then just see which ones or which combinations actually emerge as useful and reusable on our data. (In section 5.2.2 when we talk about amortization strategies this will become more plausible and appealing.) We expect this is what a system deployed in low-touch contexts will look like - equipped with a broad set of powerful primitives known to be useful for lots of potentially relevant types of data, but with few initial opinions about what to lean on. Over time, the system will learn which ones to use and how, by finding short programs that use them and short programs that reuse and recombine those in ways specifically useful to the actual data stream. Such a system will go through a long and expensive period of exploration and self-understanding before performance ramps up to steady state on the task at hand (assuming there is one, of course). The steady state resource cost can approach (or be dialed down to) the cost of the best single predictor on a validation set. This operational pattern is what motivates us to view RBII as a framework for automated engineering of streaming prediction components.

5 Applications

5.1 Continual MNIST with RBII

We now present a basic case study on a time-honored benchmark in the continual learning literature, that of continually learning an MNIST classifier. This application instantiates RBII with realistic building blocks for streaming supervised classification, while preserving the core robustness and composability guarantees implied by the universal transformer search and the reuse prior. Our aim here is NOT to represent that this is the best way to approach continual learning on this type of domain with neural-like primitives using RBII (it is almost certainly not, we believe RBII enables much more elegant solutions than the crude bias we impose here), nor is it to claim that this type of task needs or requires RBII in any conventional setting. Our aim is pedagogical - to concretely illustrate the type of thought process and workflow enabled by RBII with simple and well-known primitives on a toy example that, despite its long history in the continual learning literature, has yet to be solved in a way that presents appealing scalable properties while remaining robust over unknown and shifting input distributions.

Streaming supervised reduction. We observe a stream of labeled examples

$$(x_1, y_1), (x_2, y_2), \ldots, \qquad x_t \in \mathbb{R}^{784}, \quad y_t \in \{0, \ldots, 9\}.$$

A predictor program p induces a conditional distribution over labels given the current input and state:

$$\pi_p(\cdot \mid x_t, m_{t-1}) \in \Delta^{10}.$$

The algorithm predicts $\hat{\pi}_t(\cdot)$ as the RBII mixture over the active pool and receives log-loss $\ell_t = -\log_2 \hat{\pi}_t(y_t)$. The state update rule $m_t = \mathrm{Upd}(m_{t-1}, (x_t, y_t))$ stores history and maintains indices described below. Distribution shift corresponds to changes in the conditional $y_t \sim \mu(\cdot \mid x_t, x_{<t}, y_{<t})$ over time; in particular, earlier regimes may return after long gaps.

Fixed MLP inductive bias as a short transformer path. To make the example concrete, we fix a known-good MLP architecture for MNIST and expose it through two domain primitives:

$$\mathsf{Forward}(x, \theta) \mapsto \hat{y}, \qquad \mathsf{Backward}(F, \mathsf{loss}, x, y, \eta) \mapsto F',$$

where θ are parameters for the fixed architecture, $F : \mathbb{R}^{784} \to \Delta^{10}$ is a predictor (typically a closure around θ), loss is e.g. cross-entropy, and η is a learning rate. $\mathsf{Backward}$ returns a new predictor reflecting one gradient step. Thus a *constant-length* transformer can (i) reference the incumbent predictor and (ii) produce a slightly improved variant by a single gradient step, without any domain-specific shift detection logic. Crucially, RBII still retains universal alternatives (restarts, recall of other frozen programs, mixture/patching transformers, etc.); the NN primitives merely add a *short* path that is often useful.

5.1.1 A minimal DreamCoder-style DSL for predictors and transformers

We specify a minimal polymorphically typed lambda-calculus DSL in the DreamCoder style. The DSL contains: (i) general control/data primitives, (ii) vec-

tor/probability primitives, (iii) bounded state-query primitives (the Q interface), (iv) program reference primitives for reuse, and (v) NN bias primitives Forward/Backward. We distinguish *read-only* query primitives (available to predictors and transformers) from *write* primitives used only by the fixed update rule Upd.

Types. We treat the key domain-specific shapes as base types (as in DreamCoder practice):

$$\mathtt{X} \equiv \mathbb{R}^{784}, \qquad \mathtt{Y} \equiv \{0, \ldots, 9\}, \qquad \mathtt{Dist10} \equiv \Delta^{10}.$$

We also use:

$\mathtt{State} \equiv m$, $\mathtt{ExRef}$ (reference to a stored example), $\mathtt{ProgRef}$ (reference to a frozen program), $\mathtt{Pred} \equiv \mathtt{X} \to$

Finally, we include `Real`, `Int`, `Bool`, `Unit`, `List a`, and `Pair a b`. (Polymorphic type variables are written α, β, γ.)

Core control/data primitives (minimal).

$\mathsf{if} : \mathtt{Bool} \to \alpha \to \alpha \to \alpha$

$\mathsf{pair} : \alpha \to \beta \to \mathtt{Pair}\ \alpha\ \beta, \quad \mathsf{fst} : \mathtt{Pair}\ \alpha\ \beta \to \alpha, \quad \mathsf{snd} : \mathtt{Pair}\ \alpha\ \beta \to \beta$

$\mathsf{nil} : \mathtt{List}\ \alpha, \quad \mathsf{cons} : \alpha \to \mathtt{List}\ \alpha \to \mathtt{List}\ \alpha$

$\mathsf{map} : (\alpha \to \beta) \to \mathtt{List}\ \alpha \to \mathtt{List}\ \beta$

$\mathsf{foldl} : (\beta \to \alpha \to \beta) \to \beta \to \mathtt{List}\ \alpha \to \beta$

$\mathsf{take} : \mathtt{Int} \to \mathtt{List}\ \alpha \to \mathtt{List}\ \alpha$

$\mathsf{argmaxBy} : (\alpha \to \mathtt{Real}) \to \mathtt{List}\ \alpha \to \alpha$

Because every program is sandboxed by $\mathcal{R} = (q, T, \mathrm{mem})$, these combinators remain bounded in practice by applying them only to lists produced by bounded queries (fixed-K kNN results, small candidate lists, etc.).

Numeric/vector/probability primitives (minimal).

$(+), (-), (\times) : \mathtt{Real} \to \mathtt{Real} \to \mathtt{Real}, \quad \mathsf{log}, \mathsf{exp} : \mathtt{Real} \to \mathtt{Real}$

$\mathsf{dot} : \mathtt{X} \to \mathtt{X} \to \mathtt{Real}, \quad \mathsf{l2sq} : \mathtt{X} \to \mathtt{X} \to \mathtt{Real}$

$\mathsf{softmax} : \mathbb{R}^{10} \to \mathtt{Dist10}, \quad \mathsf{probAt} : \mathtt{Dist10} \to \mathtt{Y} \to \mathtt{Real}$

$\mathsf{xent} : \mathtt{Dist10} \to \mathtt{Y} \to \mathtt{Real}$

Here $\mathsf{xent}(\hat{y}, y)$ is cross-entropy (equivalently $-\log \hat{y}[y]$ up to log base).

State-query primitives (the Q interface). These are the only way predictors/transformers access history. Each call counts against the query budget q.

$\mathsf{knnX} : \mathtt{State} \to \mathtt{X} \to \mathtt{List\ ExRef}$ (HNSW kNN; returns fixed-K)

$\mathsf{readX} : \mathtt{State} \to \mathtt{ExRef} \to \mathtt{X}, \quad \mathsf{readY} : \mathtt{State} \to \mathtt{ExRef} \to \mathtt{Y}$

$\mathsf{knnProg} : \mathtt{State} \to \mathtt{X} \to \mathtt{List\ (Pair\ ProgRef\ Real)}$ (returns fixed-K_p)

$\mathsf{loadProg} : \mathtt{ProgRef} \to \mathtt{Pred}$ (loads a frozen predictor as a callable function)

$\mathsf{incumbent} : \mathtt{State} \to \mathtt{ProgRef}$ (reference to current best in the pool, if frozen/exposed)

Neural-network bias primitives. We expose the fixed-architecture MLP as a differentiable building block.

$$\begin{aligned}
&\texttt{Theta} \text{ (parameter bundle type for the fixed MLP)}\\
&\mathsf{initTheta} : \texttt{Unit} \to \texttt{Theta} \quad \text{(deterministic initializer)}\\
&\mathsf{Forward} : \texttt{X} \to \texttt{Theta} \to \texttt{Dist10}\\
&\mathsf{Backward} : \texttt{Pred} \to (\texttt{Dist10} \to \texttt{Y} \to \texttt{Real}) \to \texttt{X} \to \texttt{Y} \to \texttt{Real} \to \texttt{Pred}
\end{aligned}$$

Operationally, Backward performs one gradient step through the internal computation graph induced by the closure F (typically $F \equiv \lambda x.\, \mathsf{Forward}(x, \theta)$). This makes "copy incumbent + one SGD step" a constant-length transformation in the reuse language.

Write primitives (used only by Upd**, not by predictors/transformers).** To satisfy "read/write to history" while preserving the RBII sandbox model, we keep writes in the fixed update rule. Minimal writes:

$$\begin{aligned}
&\mathsf{appendEx} : \texttt{State} \to \texttt{X} \to \texttt{Y} \to \texttt{Pair State ExRef}\\
&\mathsf{hnswInsert} : \texttt{State} \to \texttt{ExRef} \to \texttt{State}\\
&\mathsf{recordIncumbent} : \texttt{State} \to \texttt{ProgRef} \to \texttt{Int} \to \texttt{State} \quad \text{(add run-length to incumbent count)}\\
&\mathsf{recordRefEdge} : \texttt{State} \to \texttt{ProgRef} \to \texttt{ProgRef} \to \texttt{State} \quad \text{("program A references program B")}\\
&\mathsf{protoIndexAdd} : \texttt{State} \to \texttt{ExRef} \to \texttt{ProgRef} \to \texttt{Real} \to \texttt{State} \quad \text{(add example} \to \text{program weight)}
\end{aligned}$$

The last three are used to build the program-recall index described next. They can be implemented either (i) by static inspection of frozen code to extract referenced `ExRef`/`ProgRef` constants, and/or (ii) by dynamic instrumentation (logging which references were touched during incumbency). Either mechanism fits the RBII model because it is part of the fixed Upd and not an unbounded retrospective validation pass.

5.1.2 State design: history + HNSW kNN + prototype→program recall

We assume m_t stores the entire example history, but predictors can only access it through a small number of kNN lookups:

$$m_t \text{ contains } \{(x_i, y_i)\}_{i \le t} \text{ and an HNSW index over } \{x_i\}_{i \le t}.$$

Thus $\mathsf{knnX}(m, x)$ returns a fixed-K list of nearby example references.

Additional program-recall index (example-conditioned code memory). To make *reuse* cheap (reducing addressing costs for long-past programs), we add a second structure: a mapping from certain "prototype" inputs to frozen program references likely to be useful near those inputs.

- Let $S_t = \{s_1, \ldots, s_{M_t}\}$ be the frozen predictor store.
- Each frozen program s_j maintains two scalar statistics in m_t:
  - **Incumbent count** $I(j)$: the total number of time steps (or total run-length) for which s_j was the incumbent/best predictor.
  - **Reference count** $R(j)$: how many other frozen programs refer to s_j (directly or via an extracted call-graph edge).
- Define a program prestige weight, e.g.

$$\mathsf{prestige}(j) \;:=\; I(j) \;+\; \lambda R(j), \qquad \lambda \in (0,1) \text{ small.}$$

Index update rule ("whenever x_k is referenced, attach it"). Whenever a program p is frozen, the update mechanism extracts the set of example references that p uses (e.g. any `ExRef` constants in p or any examples touched while p was incumbent). For each such example reference e (with vector x_e), we can radically shrink overhead

1. Mark x_e as a *prototype* if it is not already one.
2. Add/update an entry in a prototype→program adjacency structure:

$$\mathsf{Adj}[e][p] \;\mathrel{+}=\; \mathsf{prestige}(p).$$

We also maintain an HNSW index *over prototype vectors* $\{x_e : e \text{ is a prototype}\}$.

Query semantics. Given a current input x, $\mathsf{knnProg}(m, x)$:

1. Performs kNN over prototype vectors to retrieve prototypes $e_1, \ldots, e_{K_p}$.
2. Aggregates program candidates from the adjacency lists $\mathsf{Adj}[e_i]$ with a similarity kernel (e.g. $\exp(-\alpha\|x - x_{e_i}\|^2)$):

$$\mathsf{score}(p \mid x) \;:=\; \sum_{i=1}^{K_p} \underbrace{\exp(-\alpha\|x - x_{e_i}\|^2)}_{\text{input similarity}} \cdot \underbrace{\mathsf{Adj}[e_i][p]}_{\text{incumbency/ref prestige}} .$$

3. Returns the top candidates as `List (Pair ProgRef Real)`.

Why this helps. This turns long-term recall into an *input-conditioned addressing mechanism*. Rather than a transformer having to hard-code an index $\approx \log_2 |S_t|$ bits to select a past program, it can simply call $\mathsf{knnProg}(m, x)$ (constant code length) and retrieve a short candidate list. This is exactly the kind of heuristic bias that the RBII framework allows us to add while retaining universal fallback: if the heuristic is wrong, universal transformer search still enumerates other paths (including from-scratch learning and other edits).

Pruning/caching hook. Because the index stores explicit usage statistics, it also supports practical resource policies: for programs with low $\mathsf{prestige}(p)$ and low adjacency mass, we can drop cached parameter snapshots (or even drop the entire stored parameter vector) while retaining the short code description needed to re-derive or approximately reconstruct it, subject to $\mathcal{R}$.

5.1.3 Illustrative predictor/transformer patterns

Below are representative (short) programs in the DSL that RBII's universal schedule will consider. They illustrate how "known useful bias" (SGD on an MLP) is integrated as a short transformer, without excluding universal alternatives.

(1) Pure MLP predictor from scratch. A baseline predictor with freshly initialized weights:

$$p_{\text{fresh}} \;:=\; \lambda x : \mathtt{X}.\ \mathsf{Forward}(x, \mathsf{initTheta}(\star)).$$

This is always available and provides a universal fallback for new regimes: RBII can always re-learn by constructing a new parameter trajectory, even if recall heuristics fail.

(2) One-step SGD edit of the incumbent (short transformer). A transformer that takes the current state and most recent labeled example and returns a new predictor:

$$\tau_{\text{sgd}} := \lambda(m : \mathtt{State}).\lambda(x : \mathtt{X}).\lambda(y : \mathtt{Y}).\lambda(\eta : \mathtt{Real}).\ \mathsf{Backward}(\mathsf{loadProg}(\mathsf{incumbent}(m)),\ \mathsf{xent},\ x,\ y,\ \eta).$$

This is the "always a short edit exists" path: copy incumbent, apply one gradient step.

(3) Recall an old program by input-conditioned addressing. A transformer that retrieves a candidate frozen predictor specialized to inputs near x:

$$\tau_{\text{recall}} := \lambda(m : \mathtt{State}).\lambda(x : \mathtt{X}).\ \mathsf{loadProg}\big(\mathsf{fst}(\mathsf{argmaxBy}(\mathsf{snd},\ \mathsf{knnProg}(m, x)))\big).$$

This avoids hard-coding (i.e. paying for search cost of) any store index; the only "addressing" is the call to $\mathsf{knnProg}$.

(4) A cheap mixture-of-recalled-experts predictor. Given a small list of recalled program refs, form a weighted ensemble:

$$p_{\text{mix}} := \lambda(m : \mathtt{State}).\lambda(x : \mathtt{X}).\ \mathsf{let}\ C = \mathsf{knnProg}(m, x)\ \mathsf{in}\ \mathsf{let}\ C' = \mathsf{take}(K,\ C)\ \mathsf{in}\ \mathsf{normalizeMixture}\Big(\mathsf{map}(\lambda(r, s).\,(s,$$

(Here $\mathsf{normalizeMixture}$ is a tiny helper definable from foldl plus vector ops.) This is a generic compositional pattern: recall several past predictors and average them.

5.1.4 Walkthroughs: stable regimes, shifts, and returns

We sketch the qualitative RBII dynamics on three situations.

Scenario A (stable, well-predicted regime). Assume the current input distribution is stationary for long enough that the incumbent MLP p_A achieves low log-loss. Then:

- The exp-weights mixture concentrates weight on p_A (up to ϵ-smoothing).
- The short transformer τ_{sgd} occasionally generates slightly improved variants of p_A; if they produce even modest gain on the ongoing stream, they may replace the incumbent.
- Freezing stores occasional snapshots/edits of p_A (depending on the freeze policy). The prototype→program index is reinforced around examples that those programs actually reference.

Overhead is small because (i) prediction uses one dominant predictor and (ii) exploration can be kept at a minimal constant budget (details below).

Scenario B (distribution shift, sudden performance drop). Suppose the stream shifts to a new regime B (rotations, new clutter, different digit subset, label corruption pattern, etc.). Immediately:

- The mixture loss ℓ_t increases; the incumbent p_A is no longer competitive.
- RBII simultaneously explores multiple paths:
  1. *Local adaptation:* repeated applications of τ_{sgd} try to adapt p_A via SGD. This may help if B is "near" A in parameter space.
  2. *Recall:* τ_{recall} and related transformers propose old programs whose prototype neighborhoods match the new inputs.
  3. *Restart:* p_{fresh} (and multi-step variants) provide a from-scratch option if B is far.
- Online exp-weights selects whichever candidate produces lower immediate loss. If B persists, a new incumbent p_B emerges and is frozen, creating reusable structure for future returns.

Importantly, there is no brittle dependence on a single hand-tuned shift detector. If the stream is adversarial and defeats some heuristic (e.g. similarity statistics), RBII still enumerates other transformers (including restarts and alternative edits); the penalty is *time/compute*, not catastrophic permanent failure.

Scenario C (a long-past regime returns). Assume that after a long gap, regime A returns. Then:

- Many current inputs x are near prototypes that were heavily used during regime A.
- $\mathsf{knnProg}(m, x)$ returns high-scoring program refs that include (versions of) p_A, because those programs have high $\mathsf{prestige}$ and are attached to relevant prototypes.

- A short transformer re-instantiates p_A (or a close variant) and, with restart-style weight injection (as in Section 4), it re-enters the pool with nontrivial weight.
- Within a small number of steps, the mixture shifts weight back to p_A as its log-loss advantage accumulates, achieving small reacquisition delay and loss.

Tradeoffs during recovery. Even in this favorable case, RBII makes explicit tradeoffs:

- If cached parameter snapshots for p_A were pruned, reconstructing p_A exactly may exceed $\mathcal{R}$ (e.g. too many gradient replay steps). Then the recalled program may be invalid, and RBII will fall back to a cheaper approximation (e.g. a nearby cached checkpoint, a smaller edit, or re-learning).
- If the recall index returns a shortlist that omits p_A (heuristic miss), RBII still can rediscover p_A via longer addressing or alternative search paths, but the reacquisition delay grows accordingly.

5.1.5 Limiting overhead in "good" regimes

In general, RBII trades domain-coupled heuristics for resource-bound-induced biases, eschewing things like a top-level heuristic exploration policy. However if we want to reduce or eliminate the (nontrivial but still $O(1)$) exploration overhead in steady state operation, we'll need to make some additional mild assumptions about loss statistics if we wish to retain flexibility.

Is there a general-purpose notion of "good" portable across domains? Yes: in RBII the most portable notion of "good" is purely predictive/compressive. Two equivalent operational choices are:

1. **Low realized log-loss:** a recent-window average of ℓ_t is small (possibly relative to a null baseline like uniform prediction).
2. **High realized compression gain/capital:** the mixture is extracting many bits from the stream relative to a baseline, i.e. large $G_{\text{alg}}(I) = |I| - L_{\text{alg}}(I)$ or large $\mathcal{M}_{\text{alg}}(I) = 2^{G_{\text{alg}}(I)}$ on recent windows.

These notions are domain-agnostic: they require only that the interface defines a log-loss for the task outputs.

Do we need heavy exploration when performance is good? No—but we should *never* set exploration to exactly zero if we want the universal fallback property. A practical compromise is:

- Maintain a small *minimum* reminder budget $E_{\min} \geq 1$ transformer executions per step to preserve eventual discovery guarantees.
- Add an *adaptive* surplus exploration budget triggered by surprise, e.g. when a running average of ℓ_t exceeds its recent baseline or when the mixture entropy spikes.

In stable regimes, most compute goes to prediction, and exploration is dominated by short edits (e.g. SGD steps). When performance drops, exploration ramps up automatically.

Two concrete optimizations.

- **Two-tier exploration schedule.** Always run very short transformers (like τ_{sgd}) frequently; run longer/higher-variance transformers (like restarts, mixture construction, deeper edit chains) only when surprise is high, while still giving them nonzero probability overall.
- **Prestige-weighted caching.** Maintain exact parameter snapshots only for programs with high prestige and/or high adjacency mass in the prototype index; for low-prestige programs keep only code (and possibly a coarse checkpoint), accepting that exact reconstruction may fail under $\mathcal{R}$.

5.1.6 Why heuristic retrieval does not become a single point of failure

The added recall index and similarity heuristics are intentionally *not* required for correctness. They are merely a mechanism for reducing conditional description length $K_{\mathcal{R}}(\cdot \mid S; m)$ for useful programs when the current input resembles past informative prototypes. If an adversary constructs a stream that defeats these heuristics, RBII still searches over alternative transformers and can (eventually) re-learn by other routes. The failure mode is increased reacquisition delay (paying the exponential-in-description-length discovery cost), not a brittle inability to adapt.

A formal "addressing cost vs loss" inequality. The detectability/MDL lens from Section 3 makes this precise. Fix an interval I (e.g. a return episode) and two candidate predictors p and q generated by transformers of minimal lengths k_p and k_q (so $k_p \approx K_{\mathcal{R}}(p \mid S; m)$, etc.). An MDL-style selection rule favors p over q whenever

$$L_p(I) + k_p \;<\; L_q(I) + k_q,$$

equivalently

$$\underbrace{L_q(I) - L_p(I)}_{\text{loss advantage of } p} \;>\; \underbrace{k_p - k_q}_{\text{extra addressing/description cost}} .$$

This is exactly the desired statement: *a recalled past predictor becomes worth selecting (and thus worth reinstantiating with meaningful pool weight) precisely when its addressing cost is smaller than the loss paid by inferior methods on the current stream.*

In particular, the whole point of the prototype→program index is to reduce k_p for relevant old programs from $\Omega(\log |S_t|)$ (hard-coded addressing) to $O(1)$ (content-addressable recall), so that even relatively short return episodes can "pay" the description penalty and trigger rapid reacquisition.

How does the system "learn what to store"? From the point of view of a search for good predictors, a memory mechanism is something that dramatically narrows the conditional distribution of next token in the prefix-free encoding given the previous token. The universal transformer mechanism and the growing frozen store together accomplish this – freezing a program concentrates search mass on its constituents whenever a reference to that program is selected. Learning what to store in a memory to efficiently support continual learning in unknown distributions is tricky - we don't know what we should store, only that we should be storing something. Our combination of mechanisms yields an emergent index, grounded in proven predictive utility, that can leverage opinionated inductive biases (such as the specific $\mathsf{knnProg}(m, x)$ storage mechanism outlined above) without tying us to them too tightly if they turn out to be unhelpful later on.

5.2 Lessons from Program Synthesis

RBII provides a framework to deal with program learning in an online streaming prediction context. The universal nature of the framework allows us to plug in powerful techniques to speed up program search without crippling or compromising the overall system properties we desire. The DreamCoder system and its lineage have demonstrated key results directly applicable to making RBII work efficiently and effectively in practice.

5.2.1 Sophisticated Bootstrapping

DreamCoder uses a sleep-wake cycle and purpose-built refactoring algorithms to help extract and refine high-utility functions which it adds to its library of available primitives. (Later work [PLS23] also extracts useful functions by decompiling them patterns encoded in the recognition network). They demonstrate that their system is able to bootstrap sophisticated and useful primitives starting from a very minimal set in several disparate domains. Crucially, under limited training data and bounded search time, many complex programs would not be reachable without these mechanisms.

Equipping an RBII with the same mechanisms appears feasible and should yield the same benefits. The tradeoff being we spend more resources looking for deeper patterns in what we've already learned with the assumption that that is useful thing to do. This assumption appears to be widely reasonable for many domains of interest to humans, and hence likely also for generally intelligence systems, but is clearly not globally true for all sequences. The practical implication here is that even if we start out very far from a useful inductive bias for our data, given sufficient (but not astronomical) time and observations, an augmented RBII system can bootstrap a good inductive bias from simple building blocks.

5.2.2 Recursive Amortization

DreamCoder trains a recognition network on task data + discovered programs to help narrow the search space for new, relevant programs for a task. This dramatically accelerates the search process because it effectively amortizes the cost of

past successful searches by learning from them. DreamCoder however would not be expected to deal well with shifting task distribution, because it relies on batch IID training of its recognition network, which is subject to catastrophic forgetting. This is an example of how the heuristic limitations of one component directly impact the overall applicability of the system to online streaming contexts with unknown distribution shifts, albeit a mild one - if DreamCoder were directly applied to an online continual task stream, the recognition network's forgetting would hamper but not completely cripple the system as it would eventually center on a recent-enough distribution, unless of course the distribution shifts too fast for it to capture (due to misalignment with the learning rate of the network, for instance). The recognition network acts as a cache which amortizes search cost, and its not fatal if the cache happens to be cold, though it will prevent many solutions from being found that will be once it has warmed up.

Our system directly addresses this – RBII could itself manage the recognition network for DreamCoder. The basic interface is composable and the robustness to distribution shift mitigates the issue. Additionally, the RBII instance handling the recognition network itself can be completely parallelized with its client, the DreamCoder process - it only needs to make available its best predictors at any given time and observe task + program data from its client.

The same data sources that DreamCoder uses to train its recognition network also exist in our system, and a similar mechanism for biasing the search space can be employed. Further, this pattern obviously applies if RBII itself is the client, using another RBII instance to manage a recognition network (which, of course, need not be a neural network but merely any predictor that helps narrow the search space for new transformers based on past experience). That instance could in turn manage its recognition network in the same way, ad nauseam.

6 Related Work

6.1 Continual learning in modern machine learning

Most of the modern continual learning (CL) literature is motivated by catastrophic forgetting in parametric function approximators trained with stochastic gradient methods. A large body of work proposes mechanisms that preserve performance on earlier regimes while adapting to new ones, often under experimental protocols where regimes are task-labeled or at least cleanly segmented. Representative families include (i) *rehearsal and replay* methods that retain or regenerate past examples (e.g., experience replay and its variants), (ii) *regularization* approaches that penalize parameter drift deemed important to earlier tasks (e.g., elastic weight consolidation and successors), and (iii) *architectural* strategies that isolate parameters per task or expand capacity over time (e.g., progressive/dynamic networks). Surveys provide a broad taxonomy and benchmark-driven comparisons (e.g., [PKP+19, DLAM+22]).

A recurring theme, and one that motivates the present work, is that many of these systems are *mechanism-driven*: they begin with a particular learning mechanism (typically SGD on a fixed model class) and then add auxiliary heuristics—task boundary detectors, replay scheduling rules, importance-weight estimators, bespoke regularizers, buffer management, or architectural branching logic—to patch failures that arise under non-stationarity. These heuristics can be powerful within the regimes they were tuned for, but they also become coupling points: when distribution shifts are unsegmented, interleaved, or "pathological" in ways not anticipated by the designer, the auxiliary machinery can misfire, and the resulting system may be difficult to compose with other components whose own assumptions are unknown.

RBII reframes this tension by treating CL as *resource-bounded online prediction* rather than as a particular parametric training loop. Many familiar CL mechanisms can still appear as *short transformers* or *biased primitives* that accelerate search and adaptation (e.g., "copy incumbent + one gradient step" becomes a constant-length edit when exposed as a primitive), but crucially they are not load-bearing for correctness: when a heuristic fails, RBII can fall back to alternative transformers under the same universal schedule rather than becoming stuck in a brittle failure mode.

6.2 Universal induction, MDL, and resource-bounded schemes

Solomonoff induction provides an idealized solution to sequence prediction by mixing over all programs with a simplicity prior, closely related to MDL and algorithmic information-theoretic views of learning (e.g., [Sol64, LV08, Ris78]). Because the unbounded universal mixture is incomputable, many practical or semi-practical schemes introduce resource bounds via time-bounded Kolmogorov complexity, Levin-style universal search, speed priors, or structured model classes with efficient Bayesian mixtures (e.g., [Lev73, Sch02, WST95]). In reinforcement learning, Hutter's AIXI formalizes an optimal Bayesian agent under a universal prior, with practical approximations replacing the environment class by tractable mix-

tures such as (action-conditional) context tree weighting (CTW) and sampling-based planning (e.g., [Hut05, VNH+11]).

RBII is aligned with the above tradition in spirit—it keeps a universal fallback and measures progress in bits—but differs in several structural ways that matter for continual learning under tight per-step resource constraints:

- **Online mixture over an *explicit* finite active pool.** Rather than maintaining a single monolithic best hypothesis, RBII maintains a bounded set of sandbox-valid predictors and tracks them with Bayesian/exponential weights. This makes adaptation to recurring regimes a matter of reallocating weight within the pool once a good predictor is present, while discovery is handled by a separate mechanism.
- **A two-level separation between *search* and *prediction*.** RBII assigns the universal prior mass to *transformers* (program-to-program edits and generators) rather than directly to predictors. Predictors are then evaluated online by log-loss. This separation makes it possible to bound per-step prediction cost while still retaining a universal search process in the background of the algorithmic loop.
- **Explicit, composable resource accounting via a sandbox and query interface.** RBII's hypothesis class is defined operationally by a resource budget $R = (q, T, \mathrm{mem})$ and by a structured state-query interface Q that mediates access to the evolving memory. This makes compute and memory constraints first-class, rather than being implicit in an analysis that assumes unlimited evaluation or unlimited access to the raw past.

Relation to OOPS. Schmidhuber's Optimal Ordered Problem Solver (OOPS) [Sch04] is an influential incremental universal search procedure that reuses solutions to earlier problems when solving later ones. RBII shares the "incremental reuse under a universal schedule" philosophy, but targets a different setting and makes different design commitments. OOPS is primarily a solver for a *sequence of problems* where each problem has an externally defined success criterion and computation can be allocated until a solution is found. RBII instead treats continual learning as *streaming probabilistic prediction* with an irreducible, repeated requirement to produce next-step predictive distributions under a fixed sandbox budget. In RBII, reuse is mediated by *transformers* that can call/copy/edit frozen code, and usefulness is adjudicated by online log-loss within the mixture, not by a single "solved/unsolved" termination event. This makes retention and rapid reacquisition on returns a core object of analysis, rather than a byproduct of solving an ordered curriculum.

Relation to incremental compression and WILLIAM. The incremental compression viewpoint and related monolithic AGI proposals (e.g., [Fra17, FAS21, Fra25]) emphasize that compression of experience into reusable structure is a central route to data efficiency and generalization. RBII is directly inspired by this lineage and adopts an explicit compression lens (measuring improvement in bits). The distinction is that RBII operationalizes compression *under explicit per-step*

compute bounds and focuses on *contextual accessibility* of compressed knowledge: a program being present in a store is not sufficient if it cannot be retrieved or regenerated cheaply in the current context. This motivates RBII's emphasis on resource-aware conditional description length and state-indexed recall mechanisms, and it foregrounds the "addressing bottleneck" that arises when the store grows without bound.

Relation to EC/EC2 and DreamCoder-style library learning. DreamCoder and its Exploration–Compression lineage [EWN+20] demonstrate that alternating phases of program search and library refactoring can bootstrap rich, reusable abstractions that dramatically accelerate subsequent synthesis. RBII is designed so that such systems can be integrated as *heuristic accelerators*: EC-style refactoring, library induction, and recognition-model guidance can be implemented as transformer families that propose new predictors or new transformers. The key difference is that DreamCoder/EC2 are typically formulated for *batch multi-task synthesis* with an approximately stationary task distribution and an amortization mechanism (e.g., a recognition network) trained under IID-style assumptions. RBII, by contrast, is natively an *online* system for a single non-stationary stream, where amortizers are treated as caches that may go cold under distribution shift but do not compromise eventual discovery. This shifts the emphasis from "learning a DSL that best matches a task distribution" to "maintaining a universal, resource-bounded mechanism that can continually regenerate and recombine predictive code as contexts return."

Relation to AIXI with CTW and other tractable universal mixtures. CTW provides an efficient Bayesian mixture over a large class of variable-order Markov models [WST95] and has been used as a tractable environment model in AIXI approximations (e.g., [VNH+11]). RBII is compatible with this approach but not committed to it: CTW is a *particular* model class with a specialized inference algorithm, whereas RBII defines a general interface for predictors as programs with bounded access to an evolving state. In RBII, one could include CTW-like predictors as pool members, but the framework additionally supports programmatic reuse across heterogeneous predictor families via transformers and frozen code. Moreover, RBII's central continual-learning concern is not only statistical efficiency within a fixed model class, but *reacquisition efficiency* on returns when the relevant code may be long past in the store and must be retrieved under bounded compute.

Relation to other resource-bounded Solomonoff schemes. Many resource-bounded Solomonoff variants effectively trade compute for universality by penalizing runtime in the prior or by restricting the hypothesis enumeration schedule (e.g., Levin search, speed priors, time-bounded K_t-style objectives). RBII is complementary: instead of baking time into the prior over *entire predictors*, it enforces a fixed sandbox budget per invocation and places the universal prior on *edit/generation procedures* that exploit an accumulated library. This makes reuse gains *telescoping and operational*: improvements are measured by reductions in resource-aware conditional description length $K_R(\cdot \mid S; m)$, which directly translate into multiplicative

reductions in worst-case (re)generation time under the universal transformer schedule. In effect, RBII turns the classical "universal search is expensive" critique into an explicit, quantifiable engineering knob: when suitable inductive biases exist, they can be exposed as short transformers; when they do not, the system degrades gracefully by paying additional search time rather than failing outright.

6.3 More related work

RBII also connects to several adjacent traditions that address non-stationarity without assuming a fixed task segmentation.

Prediction with expert advice and tracking in changing environments. The exponential-weights mixture used by RBII is closely related to the online learning literature on prediction with expert advice, switching experts, and tracking the best expert under shifts (e.g., [CBL06, HW98]). These results provide regret guarantees relative to a *given* expert class and, with switching mechanisms, to sequences of experts. RBII can be viewed as adding a complementary piece: a universal, resource-bounded *expert discovery* mechanism via transformer search and reuse, so that the reference class itself can grow in response to experience.

Memory-augmented prediction and retrieval systems. A number of successful modern systems improve robustness to distribution shift by coupling a parametric predictor to an external memory and a retrieval mechanism (e.g., nearest-neighbor or associative lookup) that provides rapid access to relevant past items. Such systems highlight a pragmatic point that is explicit in RBII: under bounded compute, *access patterns* to memory matter as much as memory size. RBII's state-query interface Q makes retrieval a first-class object, and its analysis highlights that brittle dependence on a single retrieval heuristic can be avoided by treating retrieval itself as a non-essential accelerator: when a particular similarity metric or index fails, other transformers remain available, and the failure manifests as increased reacquisition delay rather than a hard inability to adapt.

Meta-learning, amortization, and learned search control. Meta-learning and amortized inference methods aim to speed adaptation by learning update rules or proposal distributions from experience. DreamCoder's recognition model is a concrete example in the program-synthesis setting [EWN+20]. In many CL deployments, however, amortizers are trained under stationarity assumptions that can break under continual distribution shift. RBII provides a natural "container" for amortizers: learned proposal models, replay policies, or gradient-based optimizers can be included as transformers whose utility is continuously re-validated online, while the universal search process preserves robustness when an amortizer becomes miscalibrated.

7 Limitations

In its basic form, while it will respect any asymptotic resource growth bound, the constant per-step overhead may be enormous compared to the cost of running the optimal predictor in a regime where it applies. This is due to the maintenance and evaluation of current predictors and the need to generate and test new ones. The good news is that there are many practical ways to mitigate this, as articulated in detail in section 5.1. It is worth noting here that if the deployment context distinguishes between exploration and exploitation modes at a high level, it is perfectly reasonable to throw a lot of compute at RBII to learn efficient predictors with unknown priors, then dial down the budget when the system is not expected to learn new regularities or deal with distribution shifts. A slightly more sophisticated scheme with a connection to Predictive Coding is teased briefly in 8.3

Our system cannot learn arbitrary patterns. While its scope is more general than other universal incremental learning schemes such as [Sch04] or [FGL19] in the sense that it deals with online streams of data, it will not efficiently learn to predict any sequence that has a computable predictor. The resource bounded nature of the predictors themselves and the way in which they are evaluated limits the scope to just those regularities which exhibit demonstrable information gain under resource constrained incremental search. The idea is loosely inspired by Martin-Löf's tests for computational randomness [LV08]: we can learn what compresses our stream more than the price we pay for that compression. Our resource bounds also preclude a sufficiently broad perspective for learning some complex concepts incrementally from unfriendly data streams. A blind man may eventually distinguish the correct overall shape of an elephant by repeatedly touching it in a sequence where he knows how he moved his hand each time, and can thus incrementally build a coherent model. But if those observations are scrambled, the task becomes hopeless.

8 Future Work

We now briefly outline a few more directions for future work. The number of use cases for a mature and capable CLS such as RBII is quite large, so we apologize if your favorite application is not mentioned.

8.1 Improved Formulation

While the overall scheme seems to follow naturally from our three requirements and aligns well with our observation that human learning is severely resource bounded yet universal, incremental, flexible, and highly capable of building up deep and abstract concepts, some details of the algorithm sketched in section 2 (detailed in Appendix A) and the variant in section 4 feel immature and slightly forced to us. In particular, some details around the freezing policy and management of the active pool could be improved. The use of a fixed validation window is an obvious heuristic hyperparameter we'd like to avoid without necessarily resorting to the even-more-constrained no-validation flavor of section 4. We anticipated a more refined formulation in a future iteration, but we feel the current version is sufficient to get the point across, yield useful insights under study, and seed experimental work.

8.2 Automated ML Engineering

We anticipate that RBII, with appropriate augmentations from program synthesis techniques as outlined in section 5.2.2 and 5.2.1 can already be used to explore and bootstrap novel and well-tuned learning methods for a given domain. Starting from some guesses about useful heuristics, a decent amount of uncurated data, and a whole lot of cheap(er-than-human-counterparts) compute, RBII+ should be able to discover novel essential building blocks and ways of assembling them in to competent predictors with minimal human effort.

We are particularly interested to apply RBII to the derivation of novel language models that support scalable continual learning and integration of new knowledge right out of the box. We hypothesize that such models will be easier to arrive at via human-in-the-loop RBII with healthy dose of compute than by hand-engineering methods, and that the results will involve novel and perhaps somewhat opaque combinations of neural and symbolic elements that would be difficult for a human to design from scratch.

8.3 Continual Predictive Coding in large scale systems

In a large system such as an AGI, many CLS components may interact and the system will need to decide where exploration resources should be allocated. Because RBIIs all ultimately measure performance in bits of predictive utility, an attention system where RBIIs are allocated resources based on how much they can reduce losses is a logical next step, with bits being the common denominator.

If nodes/layers in a PC network are being managed by RBII, then this becomes even more natural and appealing. In generalized PC networks, the "currency" of the

network is local prediction error. Local message passing iteratively redistributes the error among cooperating components as each component adjusts its parameters in order to output better conditional predictions of other components and/or external observations. If nodes in the PC network are managed by RBII with exploration policies that dynamically control resource allocation based on spikes in running loss statistics, an appealing dynamic picture emerges: components that can sink loss effectively (i.e. adapt well) will do so, lowering losses and loss variance, and thereby reducing their (average) resource consumption. Components that fail to decrease loss effectively will a) continue with high baseline losses, which, if they persist, also result in lower resource allocation, and b) incentivize their upstream and downstream components to redistribute that loss.

It is worth pointing out that the type of PC network just described may also be a general and robust solution to the problem posed by the author in previous experimental work on Modular PC for Continual Learning (unpublished). PC itself unlocks scalable and distributed training of neural models through local message passing, and RBII unlocks robust integration of knowledge under long term context shift. Investment in tuning RBII for the particulars of neural PC layers may therefore be highly fruitful.

8.4 Generalized Amortization via Continual Predictive Caching

Generalizing further beyond PC, a whole host of interactions between more general cognitive processes fits the pattern of figure 2: client receives (online streaming) input of unknown character, client searches for some object that satisfies some criteria over the input, and that search can be sped up by observing regularities in the results of successful or unsuccessful searches. RBII can slot in and help anywhere that the inputs and results are observable and capturable regularities can help guide the search. In the same way that RBII's operations can be amortized (section 5.2.2), it can amortize other processes in a robust way.

8.5 Cognitive Integration

Integration of DreamCoder mechanisms is featured in the paper as a mature, low-risk path to bootstrapping the program library and optimizing program search. More generally, RBII seems well-suited as an *integration substrate* for heterogeneous cognitive procedures, so long as they can be exposed through the same resource-bounded program interface: given a state-query interface Q and a sandbox budget $\mathcal{R}$, a procedure either emits candidate predictive code (new predictors, patches, abstractions) or emits intermediate structures that can be compiled into such code and then competed inside the RBII pool.

The suite of cognitive processes available in the Hyperon architecture—e.g., processes operating over a metagraph for symbolic inference, planning, and program manipulation—suggests an even richer set of integration targets. As a concrete example, a reasoning process can mine the metagraph for invariants (type constraints, causal relations, symmetries) and propose compact "checker" or "feature" programs that shrink hypothesis search and stabilize extrapolation under distribu-

tion shift; a planning/search process can generate short-horizon policies or action macros that are then frozen and indexed for rapid reacquisition when similar contexts recur; and program-evolution/refactoring processes can periodically compress frequently-used composite transformers into new reusable primitives (mirroring the spirit of section 5.2). In all cases, RBII provides a uniform acceptance criterion—improvements measured in realized predictive utility—and a conservative safety net: integrated processes may act as heuristic accelerators, but their products must earn weight in the mixture (and remain reconstructible under budget) to persist.

8.6 Cognitive Orchestration

Another direction is to view RBII's transformers not only as "patches" on predictors, but as short *control programs* that route attention across a collection of high-level cognitive processes. In this view, the active pool can contain not just candidate predictors but candidate *orchestrators*: programs whose outputs are sequences of calls to sub-processes (metagraph queries, planning loops, agentic LLM tool calls, refactoring passes, etc.) together with structured state updates. This perspective is reminiscent of blackboard and global-workspace approaches to cognitive control, where a small set of executive routines mediates access to a shared working memory and specialized solvers [HR85, Baa88].

RBII offers a natural way to *learn* such orchestration rather than hand-design it. Because every invocation is already resource-bounded by the sandbox and scored by downstream predictive gains, an orchestrator can be induced to decide when "fast" recognition-style updates suffice and when to spend additional budget on deliberative computation. For example, a learned script might (i) attempt cheap reuse via indexed recall, (ii) trigger a deeper synthesis/refinement phase only when surprise spikes or uncertainty remains high, and (iii) schedule occasional consolidation passes that refactor the library and prune brittle specialization. Mapping these roles onto Hyperon-like components suggests a clean engineering decomposition: RBII supplies the value signal and competition mechanism, while metagraph processes supply diverse proposals; the open question is how best to represent and credit-assign multi-step cognitive traces so that useful orchestration patterns become stable, reusable artifacts rather than one-off, context-fragile behaviors.

References

[Baa88] Bernard J. Baars. *A Cognitive Theory of Consciousness*. Cambridge University Press, Cambridge, 1988.

[CBL06] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

[DLAM+22] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A Continual Learning Survey: Defying Forgetting in Classification Tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 7 2022.

[EWN+20] Kevin Ellis, Lionel Wong, Maxwell Nye, Mathias Sablé-Meyer, Luc Cary, Lore Anaya Pozo, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381(2251), 6 2020.

[FAS21] Arthur Franz, Oleksandr Antonenko, and Roman Soletskyi. A theory of incremental compression. *Information Sciences*, 547:28–48, 2 2021.

[FGL19] Arthur Franz, Victoria Gogulya, and Michael Löffler. WILLIAM: A Monolithic Approach to AGI. In *Artificial General Intelligence (AGI 2019)*, volume 11654 of *Lecture Notes in Computer Science*, pages 44–58, Cham, 2019. Springer.

[Fra17] Arthur Franz. On Hierarchical Compression and Power Laws in Nature. In *Artificial General Intelligence (AGI 2017)*, volume 10414 of *Lecture Notes in Computer Science*, pages 77–86, Cham, 2017. Springer.

[Fra25] Arthur Franz. On the Atomic Structure of Information. Technical report, 2025.

[HR85] Barbara Hayes-Roth. A blackboard architecture for control. *Artificial Intelligence*, 26(3):251–321, 1985.

[Hut05] Marcus Hutter. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2005.

[HW98] Mark Herbster and Manfred K. Warmuth. Tracking the best expert. *Machine Learning*, 32(2):151–178, 1998.

[Lev73] Leonid A Levin. Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.

[LV08] Ming Li and Paul M B Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, New York, NY, USA, 3 edition, 2008.

[PKP+19] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 5 2019.

[PLS23] Alessandro B. Palmarini, Christopher G. Lucas, and N. Siddharth. Bayesian Program Learning by Decompiling Amortized Knowledge. 6 2023.

[Ris78] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 9 1978.

[Sch02] Jürgen Schmidhuber. The Speed Prior: A New Simplicity Measure Yielding Near-Optimal Computable Predictions. In Jyrki Kivinen and Robert H Sloan, editors, *Computational Learning Theory (COLT 2002)*, volume 2375 of *Lecture Notes in Computer Science*, pages 216–228, Berlin, Heidelberg, 2002. Springer.

[Sch04] Jürgen Schmidhuber. Optimal Ordered Problem Solver. *Machine Learning*, 54:211–254, 2004.

[Sol64] Ray J Solomonoff. A formal theory of inductive inference. *Information and Control*, 7:1–22, 1964.

[VNH+11] Joel Veness, Kee Siong Ng, Marcus Hutter, William Uther, and David Silver. A Monte-Carlo AIXI Approximation. *Journal of Artificial Intelligence Research*, 40:95–142, 2011.

[WST95] Frans M J Willems, Yuri M Shtarkov, and Tjalling J Tjalkens. The Context-Tree Weighting Method: Basic Properties. *IEEE Transactions on Information Theory*, 41(3):653–664, 5 1995.

A Algorithm Details

We present a more fleshed-out instance of the RBII scheme as a first step towards experimental work. We fully expect implementation work to inform a more streamlined formulation.

Resource Bounded Incremental Induction (RBII)

1: **Inputs:** update rule Upd, state-query interface Q, sandbox budget $\mathcal{R} = (q, T, \mathrm{mem})$, universal transformer schedule $\mathrm{Sched}(\cdot)$.
2: **Parameters:** pool cap N; exploration budget E_{exp} transformers/step; validation window w;
3: **Convention:** every call to a transformer or predictor is executed by the sandbox interpreter $\mathsf{U}_{\mathcal{R}}$, which enforces $\mathcal{R}$ and provides access to any state m only via Q.
4: **Initialize:** state m_0; frozen store $S_0 \leftarrow \varnothing$.
5: Insert the uniform baseline predictor $p_{1/2}$ (outputs $\pi_{1/2}(1 \mid m) \equiv \frac{1}{2}$) into the pool: $H_0 \leftarrow \{p_{1/2}\}$ with weight $W_{p_{1/2},0} \leftarrow 1$.

Candidate buffer $C_0 \leftarrow \varnothing$.
Best witnessed transformer lengths $\hat{k}(\cdot)$: set $\hat{k}(p_{1/2}) \leftarrow 0$ and $\hat{k}(p) \leftarrow +\infty$ otherwise.
Ring buffer $B_0 \leftarrow \varnothing$ storing the last w pairs (m_{t-1}, x_t) (or persistent pointers to them).
Incumbent tracker: $(p_{\text{inc}}, r) \leftarrow (p_{1/2}, 0)$.
for $t = 1, 2, \ldots$ **do**
  Predict (mixture with ϵ-smoothing)
  $Z \leftarrow \sum_{p \in H_{t-1}} W_{p,t-1}$, $\tilde{\pi}_t(1) \leftarrow \frac{1}{Z} \sum_{p \in H_{t-1}} W_{p,t-1}\, \pi_p(1 \mid m_{t-1})$, $\hat{\pi}_t(1) \leftarrow (1-\epsilon)\tilde{\pi}_t(1) + \epsilon/2$.
  Observe $x_t \in \{0, 1\}$.
  Update weights
  for each $p \in H_{t-1}$ **do**
    $W_{p,t} \leftarrow W_{p,t-1} \cdot \pi_p(x_t \mid m_{t-1})$
  Update state
  $m_t \leftarrow \mathrm{Upd}(m_{t-1}, x_t)$.
  Maintain validation window
  $B_t \leftarrow B_{t-1} \cup \{(m_{t-1}, x_t)\}$; if $|B_t| > w$ drop the oldest pair.
  Set $H_t \leftarrow H_{t-1}$ and $C_t \leftarrow C_{t-1}$.
  $H_t, C_t, p_{\text{inc}}, r \leftarrow \textsc{Explore}()(H_t, C_t, p_{\text{inc}}, r)$
  Freeze for reuse
  if $\mathrm{Check}_{\text{frz}}(p_{\text{inc}}, r, B_t)$ **then**
    $S_t \leftarrow S_{t-1} \cup \{p_{\text{inc}}\}$ (analysis often takes $\mathrm{Check}_{\text{frz}} \equiv$ true; cf. Assumption 2)
  else
    $S_t \leftarrow S_{t-1}$

Explore

function $\textsc{Explore}(H_t, C_t, p_{\text{inc}}, r)$
  Explore (universal transformer schedule):
  Draw batch $J \leftarrow \mathrm{Sched}(t)$ with $|J| = E_{\text{exp}}$.
  for each transformer $\tau \in J$ **do**
    Run τ under $\mathsf{U}_{\mathcal{R}}$ with read-only access to S_{t-1} and query access to m_t via Q.
    if τ halts with output predictor program p **then**
      $\hat{k}(p) \leftarrow \min\{\hat{k}(p), |\tau|\}$ (witness: $K_{\mathcal{R}}(p \mid S_{t-1}) \leq \hat{k}(p)$)
      if $p \notin H_t$ **and** $p \notin C_t$ **then**
        Insert p into candidate buffer C_t.
  Score candidates on the current window (MDL / detectability)
  for each $p \in C_t$ **do**
    $L_p(B_t) \leftarrow \sum_{(m_{u-1}, x_u) \in B_t} -\log_2 \pi_p(x_u \mid m_{u-1})$.
    $\mathrm{MDL}_t(p) \leftarrow L_p(B_t) + \hat{k}(p)$.
  If $|C_t| > M$, discard candidates with largest $\mathrm{MDL}_t(\cdot)$ until $|C_t| = M$.
  Select/insert (Definition 3 on the window)
  Let $A_t \leftarrow \{p \in C_t : \ \mathrm{MDL}_t(p) \leq |B_t| - \beta\}$.

for each $p \in A_t$ **(optionally capped per step) do**
  $Z \leftarrow \sum_{q \in H_t} W_{q,t}$.
  Inject weight $W_{p,t} \leftarrow 2^{-\hat{k}(p)}\, Z$ (Assumption 3)
  $H_t \leftarrow H_t \cup \{p\}$; remove p from C_t.
Evict to enforce $|H_t| \leq N$***:***
$p_{\text{old}} \leftarrow p_{\text{inc}}$.
$p_{\text{inc}} \leftarrow \arg\max_{p \in H_t} W_{p,t}$.
while $|H_t| > N$ **do**
  Remove $p^{\dagger} \in \arg\min_{p \in H_t \setminus \{p_{\text{inc}}\}} W_{p,t}$ from H_t.
Update incumbent run length
if $p_{\text{inc}} = p_{\text{old}}$ **then**
  $r \leftarrow r + 1$
else
  $r \leftarrow 1$
return H_t, C_t